A project report on

FRAUD DETECTION IN ONLINE TRANSACTIONS USING BEHAVIOURAL & COGNITIVE ANALYSIS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science Engineering

by

UTKARSH PRASAD DHEKALE (21BCE1745)

ATHARVA BODKE (21BAI1855)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November,2024

FRAUD DETECTION IN ONLINE TRANSACTIONS USING BEHAVIOURAL & COGNITIVE ANALYSIS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science Engineering

by

UTKARSH PRASAD DHEKALE (21BCE1745)

ATHARVA BODKE (21BAI1855)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2024

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled "FRAUD DETECTION IN ONLINE TRANSACTIONS USING BEHAVIOURAL AND COGNITIVE ANALYSIS" submitted by UTKARSH DHEKALE (21BCE1745), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of DR. DOMINIC M SAVIO.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: 15th November 2024 Signature of the Candidate

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"Fraud Detection in Online Transactions using Behavioural and Cognitive Analysis"** is prepared and submitted by **Utkarsh Dhekale (21BCE1745)** to Vellore Institute of Technology, Chennai, in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science Engineering** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Dominic M Savio.

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,
Bachelor in Technology Computer Science Engineering Core

Name: Dr. Nithyanandam P
Date:

ABSTRACT

The growth of online financial transactions has exponentially increased the risk of cyber threats, placing financial institutions and their customers at greater risk of fraud. Fraudulent activities lead not only to financial losses but also undermine customer trust, destabilizing the digital finance ecosystem. Traditional fraud detection methods, which largely rely on heuristic approaches, struggle to identify subtle differences between legitimate users and fraudsters. This gap in detection accuracy is becoming more pronounced as fraudsters employ increasingly sophisticated techniques, creating a pressing need for more effective and adaptive systems.

This project proposes a holistic fraud detection system designed to improve the accuracy of identifying fraudulent online financial transactions by analyzing transactional data, clickstream and telemetry data, and user persona data. The system integrates machine learning algorithms, feature engineering, data preprocessing, and model tuning to enhance fraud detection precision. By focusing on a comprehensive set of user data—such as transaction details, interaction patterns (clicks, keystrokes, session times), and device-specific information (IP addresses, browser types, device configurations)—the system captures unique patterns distinguishing genuine users from potential fraudsters.

Our approach follows a multi-step model-building process, including data collection, preprocessing, feature extraction, model training, comparison, and evaluation. The collection of transactional, clickstream, and telemetry data offers rich insights, while user persona data adds depth. Feature engineering and preprocessing ensure that the models are trained on accurate inputs, highlighting deviations from typical behaviour that are critical for detecting fraud.

The system employs machine learning algorithms tailored to process and analyze the data in real-time. These models are trained to detect both subtle and overt signs of fraudulent behaviour, making it more difficult for fraudsters to mimic legitimate users. Through continuous evaluation and model tuning, the system minimizes false positives while maintaining high detection accuracy.

By integrating diverse data sources into a unified, real-time fraud detection system, this approach provides a more comprehensive understanding of user behaviour, making it harder for fraudsters to exploit vulnerabilities in traditional systems. The holistic view captured by this system allows for more accurate identification of fraudulent transactions, improving online financial security.

This project aims to enhance fraud detection systems, protect user data and funds, and restore customer trust in digital finance. Through innovative use of behavioural, cognitive, and transactional data, the system represents a significant step forward in the fight against online fraud, offering financial institutions a powerful tool to prevent fraud in real-time.

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to Dr Dominic M Savio, Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, and understanding; more than all, he taught me patience in my endeavour. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of Machine Learning.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr G. Viswanathan our Honorable Chancellor, Mr Sankar Viswanathan, Dr Sekar Viswanathan, Dr G V Selvam Vice Presidents, Dr Sandhya Pentareddy, Executive Director, Ms Kadhambari S. Viswanathan, Assistant Vice-President, Dr V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.
Special mention to Dr. Ganesan R, Dean, Dr. Parvathi R, Associate Dean Academics, Dr. Geetha S, Associate Dean Research, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In a jubilant state, I express ingeniously my whole-hearted thanks to Dr. Nithyanandam P, Head of the Department, B.Tech. Computer Science Engineering Core and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staff at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: 15th November 2024 **Utkarsh Prasad Dhekale**

CONTENTS

CONTENTS iv

LIST OF FIGURES ix

LIST OF TABLES xi

LIST OF ACRONYMS xii

CHAPTER 1 INTRODUCTION

1.1 ONLINE PAYMENTS 11

1.2 FRAUD DETECTION SYSTEMS 11

1.3 CURRENT SYSTEMS 12

1.4 LIMITATIONS 13

1.5 PROBLEM STATEMENT 14

1.6 RESEARCH OBJECTIVES 14

1.7 SCOPE OF THE PROJECT 15

CHAPTER 2 OUR SOLUTION

2.1 PROPOSAL 16

2.2 ARCHITECTURE 19

2.3 DATA COLLECTION 22

2.4 TRADITIONAL ML MODELS 24

2.5 FEATURE EXTRACTION 32

2.6 DIMENSIONALITY REDUCTION 33

2.7 CUSTOM CLUSTERING MODEL 34

2.8 RESULTS & OBSERVATION 44

CHAPTER 3 FUTURE SCOPE

3.1 PROTOTYPE CONSTRAINTS…….47

3.2 PAYFLOW INTERFACE……..47

3.3 FUTURE SCOPE……48

3.4 TECHNICAL ADVANCEMENTS……49

CHAPTER 4 CONCLUSION

4.1 PERSONAL LEARNINGS & GROWTH……..…….51

4.2 CONCLUSION……...52

APPENDICES 53

REFERENCES 54

LIST OF FIGURES

2.1 PROPOSED SOLUTION 18

2.2 ARCHITECTURE DIAGRAM 21

2.3 TRANSACTION DATA HEATMAP 25

2.4 TRANSACTION DATA DECISION TREES 26

2.5 CONFUSION MATRIX 31

2.6 DISTRIBUTION OF CONTINUOUS VARIABLES 35

2.7 TRANSACTION DISTRIBUTION TYPE 36

2.8 EVENT DISTRIBUTION TYPE 36

2.9 TRANSACTION AMOUNT BY FRAUD STATUS 37

2.10 EVENT DISTRIBUTION TYPE 37

2.11 EXPLAINED VARIANCE RATIO OF PCA COMPONENTS 38

2.12 CLUSTERING FLOWCHARTS 41

2.13 PLOTTED GRAPHS 42

2.14 DEMO PAYFLOW INTERFACE 47

LIST OF TABLES

2.1 TRANSACTION DATASET 24

2.2 TRANSACTIONAL RANDOM FOREST ACCURACY 26

2.3 TRANSACTIONAL XGBOOST ACCURACY 26

2.4 CLICKSTREAM DATASET 27

2.5 CLICKSTREAM RANDOM FOREST ACCURACY 28

2.6 CLICKSTREAM SGBOOST ACCURACY 28

2.7 CLICKSTREAM SVM ACCURACY 29

2.8 USER PROFILE DATASET 29

2.9 USER RANDOM FOREST ACCURACY 30

2.10 USER XGBOOST ACCURACY 30

2.11 USER SVM ACCURACY 31

2.12 FEATURE EXTRACTION 32

2.13 DIMENSIONALITY REDUCTION 33

2.14 CUSTOM MODEL ACCURACY 45

Chapter 1
Introduction

1.1 ONLINE PAYMENTS

Online payments have become a cornerstone of the modern digital economy, offering convenience, speed, and global accessibility for both businesses and consumers. As a new-age solution, online payment platforms facilitate seamless transactions, allowing users to purchase goods and services instantly through their smartphones, computers, and other connected devices. However, with this rapid shift towards digital payments comes a significant risk: the rising threat of fraud. Fraudsters exploit vulnerabilities in the online payment ecosystem, using techniques like phishing, identity theft, and fake accounts to gain unauthorized access to user funds and personal information. The rich data available on payment platforms, such as user data (email addresses, phone numbers), transactional data (purchase amounts, payment methods), and clickstream data (mouse movements, clicks, session duration), presents both an opportunity for enhancing fraud detection and a target for malicious actors. This data provides insights into user behaviour, device details, and transaction history, which, if compromised, could lead to substantial financial losses. Therefore, securing this data and detecting fraudulent activity in real-time has become a critical challenge for online payment providers, requiring advanced fraud detection systems that can process and analyze this data to identify and prevent fraud.

1.2 FRAUD DETECTION SYSTEMS

Fraud detection systems are advanced tools designed to identify and prevent fraudulent activities across various platforms, particularly in financial transactions. These systems use a combination of machine learning algorithms, behavioural analytics, and real-time monitoring to detect suspicious activities, such as unauthorized transactions, identity theft, and account takeovers. By analyzing large volumes of transactional data, user behaviour patterns, and historical trends, fraud detection systems can flag unusual activities that deviate from typical behaviour, alerting financial institutions and users before significant damage is done. Their scope extends beyond just monitoring for fraud; they also include features like risk scoring, predictive analytics, and transaction validation to assess the likelihood of fraud in real-time. With the rise of digital transactions, these systems are increasingly vital for securing online payments, protecting customer data, and ensuring financial integrity. They play a critical role in minimizing financial losses, reducing false positives, improving the customer experience, and safeguarding the trust and stability of financial systems in the face of growing cyber threats.

1.3 CURRENT SYSTEMS

1. **Behaviour-Based Fraud Detection**:
    - *Wang and Zhu (2022)* highlight the significance of capturing detailed behavioural patterns through fine-grained co-occurrences to enhance fraud detection in online payment services.
    - *Ruan et al. (2016)* focus on individual and population-level models that use spatiotemporal and location-based data to detect behavioural anomalies.
    - *Zhang, Zheng, and Min (2018)* emphasize the importance of sequential behavioural data using deep learning and Markov Transition Fields to identify online fraud.
    - *Min et al. (2021)* explore the use of explainable behavioural sequence clustering for fraud detection, highlighting the value of analyzing behavioural sequences.
2. **Network and Social Feature-Based Detection**:
    - *Wang et al. (2012)* propose using social network structures for email classification, classifying emails based on social features without relying on content, which has implications for fraud detection.
    - *Hofgesang and Kowalczyk (2005)* analyze clickstream data and focus on detecting anomalies through communication network analysis.
3. **Big Data and Real-Time Analysis**:
    - *Vanini et al. (2023)* discuss transitioning from anomaly detection to a risk management approach, stressing the need for real-time data handling for online payment fraud.
    - *Olabiyi et al. (2024)* highlight the importance of real-time transaction analysis to detect fraud as quickly as possible, enabling continuous monitoring.
    - *Kian and Obaid (2022)* investigate the use of big data clustering techniques and customer behaviour indicators to enhance fraud detection for banking transactions.
4. **Machine Learning and Advanced Techniques**:
    - *Dornadula and Geetha (2019)* provide a comprehensive review of machine learning algorithms in credit card fraud detection, emphasizing the potential of machine learning to improve detection accuracy.

1.4 LIMITATIONS

1. **Behavioural Data Quality**:
    - Many studies indicate that low-quality or insufficient behavioural data limits detection effectiveness, particularly for low-frequency behaviours, making it difficult to accurately identify fraud patterns (Wang and Zhu, 2022; Ruan et al., 2016).
2. **Data Representation Limitations**:
    - Current models struggle to capture detailed co-occurrences between transactional attributes, which are crucial for detecting subtle fraudulent activities (Wang and Zhu, 2022; Min et al., 2021).

3. **Scalability Challenges**:
    - Handling large datasets in real-time poses a significant challenge, reducing the applicability of many methods in real-world systems where large-scale data processing is required (Vanini et al., 2023; Olabiyi et al., 2024).
4. **Inadequate Communication Network Utilization**:
    - Many studies fail to integrate social and communication network structures effectively, which could provide context in detecting fraud or classifying unusual interactions (Wang et al., 2012; Hofgesang and Kowalczyk, 2005).

Defining the Research Gaps

1. **Fine-Grained Co-Occurrences**:
    - There is a need for methods that model detailed attribute-level co-occurrences within transactional data. This gap represents an opportunity to enhance behavioural models for fraud detection (Wang and Zhu, 2022; Min et al., 2021).
2. **Contextual Email Classification**:
    - Existing studies lack a focus on email classification within the broader context of written exchanges, a gap that is relevant for systems integrating communication data with behavioural analysis (Wang et al., 2012).
3. **Real-Time and Big Data Handling**:
    - Improved real-time analysis techniques are needed for efficiently managing large datasets, which remains challenging for most current approaches (Vanini et al., 2023; Olabiyi et al., 2024).
4. **Machine Learning Adaptation**:
    - Further research is needed to adapt advanced machine learning methods to address issues of accuracy and scalability, essential for robust fraud detection models (Dornadula and Geetha, 2019).

Importance of Addressing the Gaps

1. **Enhanced Fraud Detection Accuracy**:
    - Modeling fine-grained co-occurrences could significantly improve the accuracy of fraud detection in real-time payment systems by identifying subtle behavioural nuances (Wang and Zhu, 2022).
2. **Understanding Communication Influence**:
    - Addressing the gap in contextual email classification could lead to a deeper understanding of interpersonal influence in communication, enhancing classification systems (Wang et al., 2012).
3. **Real-Time Fraud Detection**:
    - Improving scalability and real-time analysis will make fraud detection systems more responsive and reliable, essential for the integrity of online payment networks (Vanini et al., 2023; Olabiyi et al., 2024).
4. **Scalable Machine Learning Solutions**:
    - Enhancing machine learning scalability will support various fraud detection scenarios, creating more adaptable and robust models for industry applications (Dornadula and Geetha, 2019).

By addressing these limitations, the project seeks to contribute to a more integrated, comprehensive fraud detection solution that leverages behavioural data, social networks, and advanced machine learning techniques.

1.5 PROBLEM STATEMENT

Despite advancements in fraud detection systems, existing methods still face significant limitations that hinder their effectiveness. One of the key challenges is that many fraud detection systems focus on analyzing isolated data domains. For example, transactional data may be analyzed separately from behavioural data, such as clickstream activity or user interactions. While this allows for the detection of specific fraud patterns, it fails to account for the broader context of user behaviour. As a result, fraudsters can exploit gaps in these fragmented systems, making it easier for them to bypass traditional detection methods. A lack of integration between different data types limits the system's ability to detect subtle and complex fraud patterns that may span multiple domains, reducing its overall accuracy and effectiveness.

To address these limitations, a holistic approach to fraud detection is required. This approach involves the integrated analysis of various data sources, including transactional data, clickstream behaviour, and additional user insights such as emails and phone interactions. By combining these different data streams, fraud detection systems can develop a more comprehensive view of user behaviour, providing a deeper understanding of what constitutes legitimate versus fraudulent activity. A holistic solution not only enhances the system's ability to detect sophisticated fraud attempts but also improves accuracy by eliminating blind spots that individual data domains might create. This comprehensive analysis allows for more nuanced fraud detection, ensuring that patterns of fraud that span multiple sources can be identified more effectively and in real-time. Through this integrated approach, institutions can better safeguard against fraud while minimizing false positives and improving overall system reliability.

1.6 RESEARCH OBJECTIVES

This research aims to develop an advanced fraud detection system capable of identifying and preventing fraudulent activities in online transactions by leveraging a combination of transactional, user persona, behavioural, and cognitive data. To achieve this, the project focuses on the following key objectives:

1. **Detecting and Preventing Online Fraud through Multi-Domain Data Analysis:** The primary objective is to enhance fraud detection by integrating various data types—transactional, user persona, behavioural, and cognitive. This holistic analysis allows the system to gain a comprehensive understanding of user behaviour and detect nuanced fraud patterns that may not be identifiable through isolated data streams.

2. **Developing a Technical Proof of Concept Using Advanced Machine Learning Models:** To ensure the system's effectiveness, the project will implement a range of machine learning algorithms, such as XGBoost, Random Forest, and Support Vector Machines (SVM). If these standard models prove insufficient, custom clustering algorithms will be developed to improve classification accuracy in detecting fraud.

3. **Optimizing Detection through Feature Extraction and Dimensionality Reduction:** The project seeks to improve detection accuracy and reduce false positives by applying feature extraction and dimensionality reduction techniques. These techniques will help refine raw data into meaningful features, optimizing the input for machine learning models and enhancing the system's performance without overwhelming computational resources.

4. **Assigning an Optimized Fraud Score for User Risk Assessment:** To evaluate the risk of fraud at the individual level, the system will assign an optimized fraud score to each user. This score will be calculated based on a holistic analysis of the user's activity, indicating the likelihood of fraudulent behaviour. This fraud score allows for real-time risk assessment, enabling financial institutions to prioritize high-risk cases and minimize the impact of fraud.

1.7 SCOPE

This research focuses on developing an advanced fraud detection system for online financial transactions, specifically leveraging user behaviour patterns on payment flows to detect fraudulent activity. The scope of this project includes:

- **Behavioural Analysis in Online Financial Transactions:** The project targets online financial transactions where user interactions on payment platforms, known as payflow behaviour, can be observed and analyzed. By tracking patterns such as click frequency, session duration, and navigation paths, the system will identify deviations that could indicate fraud.

- **Machine Learning for Fraud Prediction:** Machine learning models will be applied to process large datasets, drawing from behavioural and cognitive data to generate accurate fraud predictions. This approach enables real-time detection of anomalous activity, providing timely intervention in potential fraud cases.

Chapter 2
Solution

2.1 PROPOSAL

Our Proposal: A Holistic Fraud Detection System

To address the complexities and limitations identified in existing fraud detection systems, we propose a holistic, multi-dimensional approach to detecting online fraud. Our system is designed to provide an in-depth analysis by examining three primary data categories: Transactional Data, Clickstream and Telemetry Data, and User Persona Data. By integrating these distinct data streams with advanced machine learning techniques, our model aims to accurately detect fraudulent activity with an emphasis on minimizing false positives and maximizing detection rates.

Core Components of Our Holistic System

1. **Transactional Data Analysis:**
   - Our model leverages detailed transactional records, which include transaction amounts, timestamps, account balances, and geolocation information.
   - By identifying unusual transaction behaviours, such as deviations from the user's typical spending patterns or rapid successive transactions, the model can flag activities that fall outside of normal behaviour.

2. **Clickstream and Telemetry Data Analysis:**
   - Clickstream data provides a granular view of user interactions on the platform, capturing details like page views, button clicks, time spent on various sections, and navigation patterns.
   - Telemetry data encompasses technical information related to the user’s device and network, such as IP addresses, device type, and connection quality. This data is particularly useful in detecting abnormal access patterns, like sudden changes in IP addresses or unfamiliar devices accessing a user account.
   - By combining clickstream and telemetry data, our system aims to identify unique user behaviours and potential anomalies that may indicate unauthorized access or suspicious activities.

3. **User Persona Data Analysis:**
   - This layer involves examining personal and behavioural data, such as demographics, historical account behaviours, and past interaction frequencies.
   - User persona data enables the model to understand an individual's regular patterns and allows for better differentiation between genuine user behaviour and potentially malicious activity.
   - Incorporating user-specific details allows our model to customize fraud detection thresholds and refine its accuracy over time based on the user’s unique behavioural fingerprint.

Integrating Machine Learning Techniques

To leverage the insights from each data category effectively, our system incorporates machine learning algorithms optimized for fraud detection. This includes:

1. **Feature Engineering:**
   - Through feature engineering, we extract relevant attributes from the raw data, transforming transactional records, clickstream logs, and persona details into machine-readable features.
   - For example, we may create features that capture the frequency of specific actions, identify high-risk geolocations, or monitor login frequency from new devices. These derived features enhance the model's ability to detect fraud patterns that might not be obvious from raw data alone.

2. **Data Preprocessing:**
   - The preprocessing phase involves cleansing and normalizing the data, handling any missing or anomalous entries, and preparing it for machine learning. Techniques such as data normalization, outlier removal, and categorical encoding ensure that the dataset is optimal for accurate and efficient processing.

3. **Model Tuning and Training:**
   - Our approach involves training a blend of traditional algorithms (e.g., XGBoost, Random Forest, SVM) alongside custom clustering techniques to evaluate the efficacy of various models in detecting anomalies.
   - Model tuning is carried out through hyperparameter optimization to achieve the best possible balance between recall and precision, thereby reducing the rate of false positives and enhancing the system's fraud detection capabilities.

4. **Multi-Stage Model Evaluation:**
   - We conduct a comprehensive evaluation to compare the performance of different models. Each model's output is rigorously tested against real-world and simulated fraud scenarios to ensure robustness.
   - After the comparison phase, the most accurate model configuration is selected and subjected to continuous testing and feedback loops, allowing for iterative improvements as new fraud trends emerge.

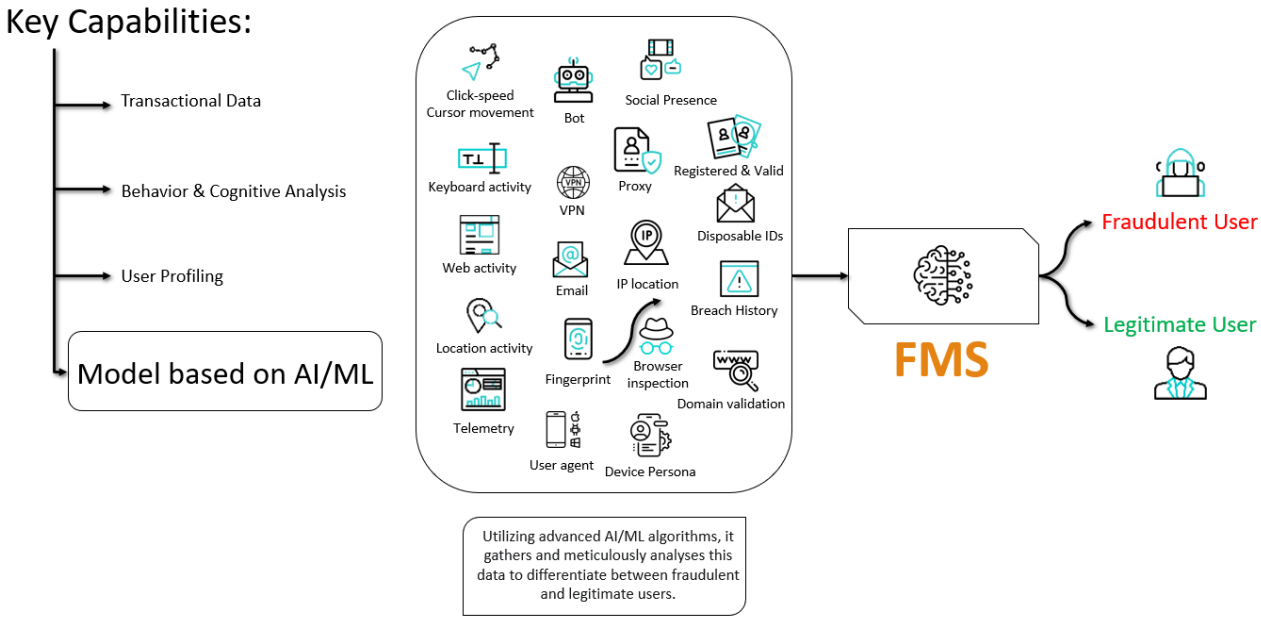

Fig. 2.1 Proposed Solution

Advantages of Our Holistic Approach

1. **Real-Time Fraud Detection:**
   - By integrating clickstream and telemetry data, our system can detect potentially fraudulent activity in real-time, allowing for immediate response measures. This is particularly advantageous in fast-paced online payment environments where timely detection is crucial.

2. **Contextualized Anomaly Detection:**
   - Through its multi-faceted data analysis, our model captures a more comprehensive view of user behaviour. Rather than focusing on isolated data points, it assesses patterns in a contextualized manner, making it harder for fraudsters to mimic genuine user activity.

3. **Scalability and Adaptability:**
   - Our system is designed to handle large datasets and adapt to new data points dynamically, making it scalable for application in environments with heavy transaction volumes.
   - The model's adaptability ensures that it remains effective as fraud patterns evolve, addressing issues of outdated fraud detection techniques and providing long-term security benefits.

4. **Minimized False Positives:**
   - By capturing unique behavioural patterns and fine-tuning model parameters, our system minimizes false positives, reducing disruptions to legitimate users while accurately identifying fraudulent actions.

Model Implementation Workflow

Our holistic fraud detection system follows a structured, multi-step model-building process that encompasses the following stages:

1. **Data Collection:**
   - Aggregate diverse data from transactional records, clickstream logs, and user personas, ensuring data completeness and quality across all sources.

2. **Preprocessing and Feature Extraction:**
   - Clean, normalize, and encode data while extracting key behavioural features to maximize machine learning efficiency.

3. **Model Training and Customization:**
   - Train multiple models using traditional algorithms and custom clustering techniques, iteratively refining based on performance.

4. **Evaluation and Optimization:**
   - Conduct rigorous testing and comparisons across model outputs, fine-tuning parameters to achieve optimal fraud detection accuracy.

5. **Final Deployment and Monitoring:**
   - Deploy the selected model in a real-world environment with ongoing monitoring to continuously enhance its detection performance and adapt to new fraud strategies.

By implementing this holistic fraud detection system, we aim to establish a comprehensive solution that not only detects fraudulent activities with high precision but also provides a scalable and adaptable framework suited for the evolving landscape of online finance. This system stands as a robust, proactive measure for financial institutions to ensure secure and trustworthy online transactions for their users.

2.2 ARCHITECTURE

This architecture illustrates a comprehensive fraud detection system designed to analyze and score online transactions for potential fraudulent activity. The system follows a multi-step approach to identify and label transactions based on various data types and machine learning models. Here's an overview of each section:

1. **Data Collection:** Data is gathered from multiple sources through APIs or web crawlers, which serve as entry points for collecting diverse information on users and transactions.

2. **Multimodal Data Sources:** The data is divided into three main categories:
   - Transactional Data: Includes account details, transaction amount, timestamp, and payment modes, which provide insight into user financial behaviour.
   - Behavioral & Cognitive Data: Contains clickstream information, cursor movement,

web activity, location, and other user interactions, which help detect unusual patterns indicative of fraud.

- User Persona Data: Comprises user identification details, device information, social connections, and contact methods, aiding in validating the legitimacy of a user.

3. **Feature Engineering:**

- Feature Extraction: Processes raw data to derive useful features (X) that capture essential transaction and user behaviour details.
- Dimensionality Reduction: Reduces the number of features from X to a smaller subset (Y) to improve model performance by eliminating irrelevant data.

4. **Traditional ML Model:** The dataset is passed to various machine learning models tailored for different data types:

- Transaction Model: Uses algorithms like Random Forest, XGBoost, and SVM to analyze transactional data.
- Behavioral & Cognitive Model: Leverages neural networks and decision-tree-based models to detect behavioural anomalies.
- User Persona Model: Employs Bayesian classifiers and logistic regression to validate identity-related attributes.

5. **Custom Classification and Clustering Model:** The refined set of features (Z) is then processed through a custom classification or clustering model to identify and classify potentially fraudulent behaviour. If traditional classification algorithms (such as Random Forest or SVM) do not provide the required accuracy, custom clustering techniques are employed to better capture complex patterns. Key steps include:

    - Classification: Attempts to label user behaviours based on predefined fraud/non-fraud classes.
    - Clustering: Groups similar behaviours to detect outliers, which may represent anomalous or fraudulent activities.

    Clustering allows the model to identify irregular patterns by grouping typical user behaviours together and highlighting outliers—instances that deviate significantly from expected patterns. This approach helps in detecting sophisticated fraud tactics that may bypass traditional rule-based or classification-only systems. Together, these steps form a robust feature engineering process that enhances the model's ability to distinguish genuine users from potential fraudsters with high precision and minimal latency.

5. **Optimization:** A Fraud Management System integrates the outputs from each model, optimizing them to produce a consolidated fraud score. This optimization process improves the system's decision-making accuracy.

6. **Comparator and Decision Layer:**

- The fraud score generated by the models is evaluated against a threshold in the Comparator.
- Based on the score, the transaction is classified as either Fraud or Normal.

7. **Output Layer:** The system finalizes the fraud prediction, labelling users as either fraudulent or legitimate based on the optimized score. This prediction can then be used to prevent or flag fraudulent activities, ensuring secure transactions.

This architecture combines transactional, behavioural, and personal data to form a robust fraud detection system that leverages advanced machine learning models and data processing techniques to detect sophisticated fraud patterns in real-time.

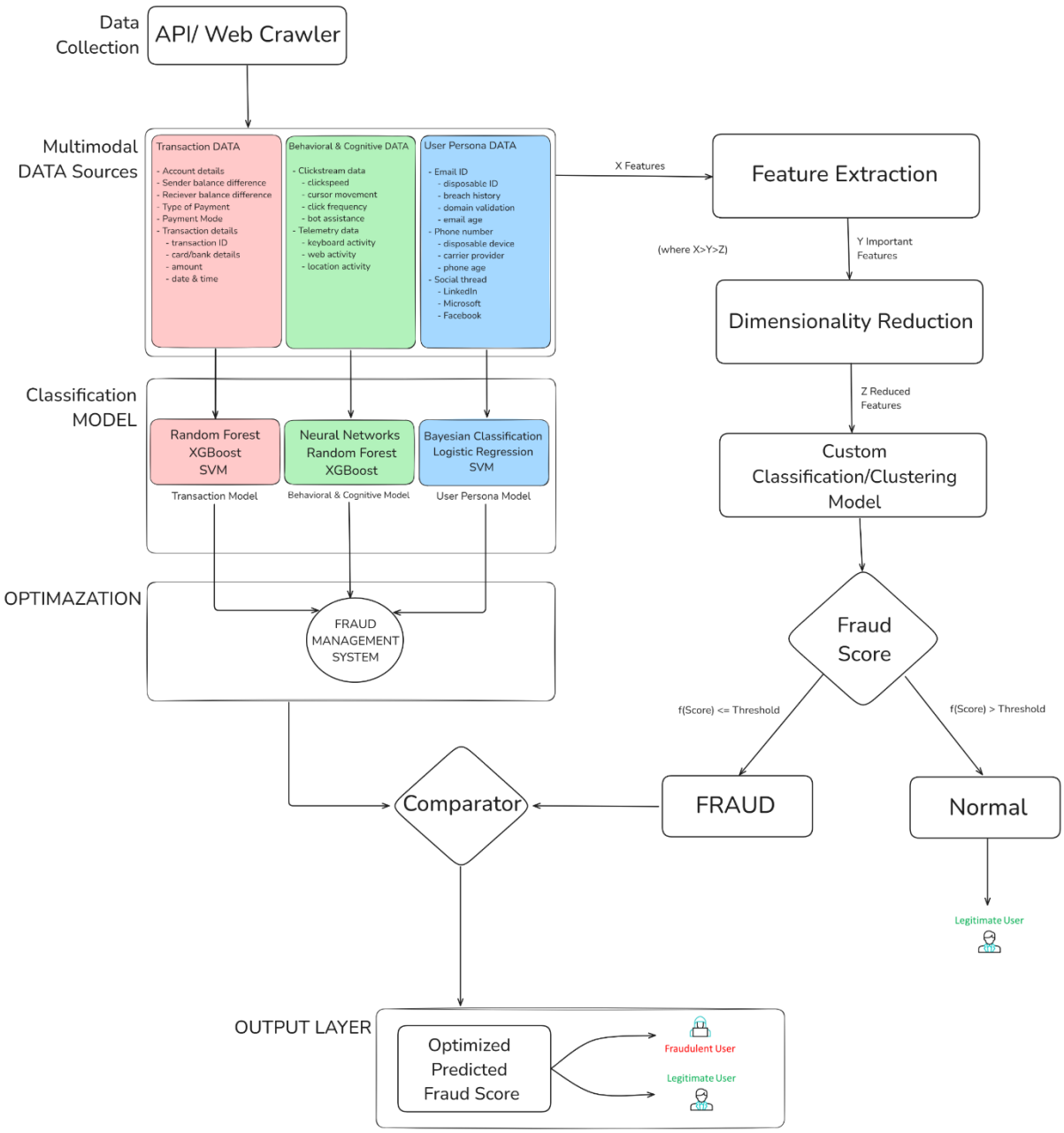

Fig 2.2 Architecture diagram

2.3 DATA COLLECTION

For effective fraud detection, the system requires multiple data types to capture a comprehensive view of user behaviour and transaction context. **Transactional Data** provides insight into financial details like transaction amounts and bank records. **Clickstream Data** tracks user interactions such as clicks, keystrokes, and session duration, highlighting engagement patterns. Telemetry Data offers contextual information, including device type, IP address, and browser details, which can reveal unusual access methods. Finally, **User Profile Data** encompasses attributes like email validation status, breach history, and age, which help verify user authenticity. This diverse, high-resolution data is essential for identifying subtle distinctions between legitimate and fraudulent behaviours, thereby enhancing the accuracy of the detection system.

Transactional Data:


```
{"user_id": "abc",
   {
      "txnID": "TX123456789",
      "type": "TRANSFER",
      "amount": 1500.75,
      "nameOrig": "JohnDoe123",
      "oldbalanceOrg": 5000.00,
      "newbalanceOrig": 3500.00,
      "nameDest": "JaneDoe456",
      "oldbalanceDest": 2000.00,
      "newbalanceDest": 3500.75,
      "isFraud": false,
      "isFlaggedFraud": false,
      "transactionCategory": "payment",
      "currency": "USD",
      "merchant": “Amazon",
      "isInternational": false,
      "transactionFee": 3.50
   },
}
```


Clickstream Data:


```
{"user_id": "abc",
   {"sid": 1,
      {
         "timestamp": 60392234,
         "event": "click",
         "pos_x": 10,
         "pos_y": 20,
         "distance": "3cm",
         "interval": "3s",
         "speed": "1cm/s"
```

```
        }
        {
            "timestamp": 60392237,
            "event": "scroll",
            "pos_x": 20,
            "pos_y": 50,
            "distance": "9cm",
            "interval": "3s",
            "speed": "3cm/s"
        };
        "SessionFraud": 1,
    };
    "UserFraud": 1
}
```


User Profiling Data:


```
{"user_id": "abc",
        0022name": "John",
        "age": 30,
        "email":{
            "id": "john@gmail.com",
            "domain": "gmail",
            "breach history": 3,
            "isValid": true,
            "domainVaildation": true,
            "Microsoft": false,
            "LinkedIn": true,
            "Meta": true,
            "Disposable_Id": false,
            "emailAge": 3
        },
        "phone":{
            "number": "754896231",
            "carrier": "Airtel",
            "diposableSim": false,
            "phoneAge": 1
        },
    },
}
```

2.4 TRADITIONAL MACHINE LEARNING MODELS

To begin our approach, we implemented traditional machine learning models on individual datasets, replicating the existing methods used in fraud detection systems and assessing their outputs. This initial stage involved applying standard models like Random Forest, SVM, and XGBoost separately on each data domain to evaluate their predictive capabilities.

Transactional Dataset:
Our first attempt at building a fraud detection model focused on the transactional dataset. By training traditional ML algorithms on this dataset, we aimed to capture transaction-specific patterns such as unusual transaction amounts, frequency, and timing, which are common indicators of fraud.

In [3]:
```
df1=df
df.head(10)
```

Out[3]:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.00	160296.36	M1979787155	0.0	0.00	0	0
1	1	PAYMENT	1864.28	C1666544295	21249.00	19384.72	M2044282225	0.0	0.00	0	0
2	1	TRANSFER	181.00	C1305486145	181.00	0.00	C553264065	0.0	0.00	1	0
3	1	CASH_OUT	181.00	C840083671	181.00	0.00	C38997010	21182.0	0.00	1	0
4	1	PAYMENT	11668.14	C2048537720	41554.00	29885.86	M1230701703	0.0	0.00	0	0
5	1	PAYMENT	7817.71	C90045638	53860.00	46042.29	M573487274	0.0	0.00	0	0
6	1	PAYMENT	7107.77	C154988899	183195.00	176087.23	M408069119	0.0	0.00	0	0
7	1	PAYMENT	7861.64	C1912850431	176087.23	168225.59	M633326333	0.0	0.00	0	0
8	1	PAYMENT	4024.36	C1265012928	2671.00	0.00	M1176932104	0.0	0.00	0	0
9	1	DEBIT	5337.77	C712410124	41720.00	36382.23	C195600860	41898.0	40348.79	0	0

Table 2.1 Transactional Dataset

In [9]:
```
def detect_outliers(series):
    q1=series.quantile(0.25)
    q3=series.quantile(0.75)
    iqr=q3-q1
    lower_bound=q1-1.5*iqr
    upper_bound=q3+1.5*iqr
    outliers=(series<lower_bound)|(series>upper_bound)
    return outliers.sum()
for x in df.columns:
    if df[x].dtype=="int64" or df[x].dtype=="float64":
        num_outliers = detect_outliers(df[x])
        print(f"Number of outliers in column '{x}': {num_outliers}")
```

```
Number of outliers in column 'step': 102688
Number of outliers in column 'amount': 338078
Number of outliers in column 'oldbalanceOrg': 1112507
Number of outliers in column 'newbalanceOrig': 1053391
Number of outliers in column 'oldbalanceDest': 786135
Number of outliers in column 'newbalanceDest': 738527
Number of outliers in column 'isFraud': 8213
Number of outliers in column 'isFlaggedFraud': 16
```

```
In [16]: import seaborn as sns
         sns.heatmap(df.corr())

Out[16]: <Axes: >
```

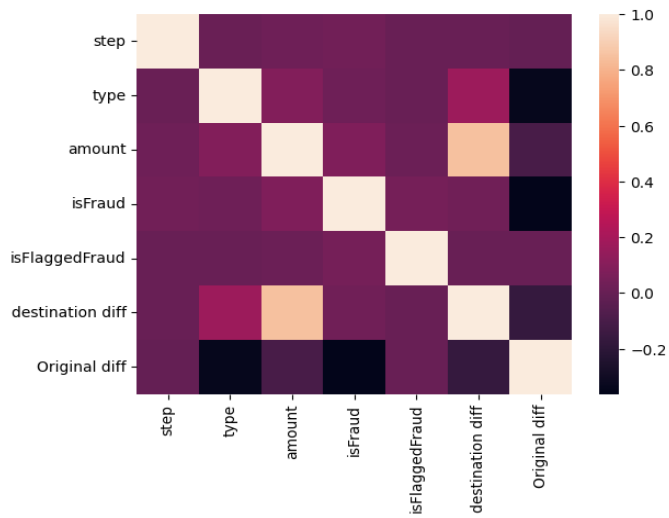

Fig 2.3 Transaction data heatmap

Random Forest:

```
In [18]: import pandas as pd
         from sklearn.model_selection import train_test_split
         from sklearn.ensemble import RandomForestClassifier
         from xgboost import XGBClassifier
         from sklearn.svm import SVC
         from sklearn.metrics import accuracy_score, classification_report


         # since there are many outliers in the important features, I have tried using three ML models which are not as sensitive
         # compared to other models. Removing outliers in cases where there are binary classes will affect the entire prediction
         # and they cannot be exchanged with the mean,median or mode.


         independent=["type","amount","destination diff","Original diff"]

         # I have selected the above features for prediction because name of the source/destination does not affect the decision
         # of a transaction being fraud or not. Features like total amount transacted, and the resultant balance in original account
         # will allow us to understand if its a fraud or not. Even the difference in the balance of the destination account. And many
         # times there are cases where the transaction type and how long it took leads to understanding if its a fraud or not. Eg:
         # if a transaction is taking long then the destination must be well hidden.

         df_subset = df1.sample(frac=0.1, random_state=42)

         # due to large data size, I have tried random sampling to see how it works. Due to need of more storage I shifted to kaggle
         # to use its storage and GPU power. The accuracy remains the same even when applying it to the entire dataset.

         X_subset = df_subset[independent]
         y_subset = df_subset["isFraud"]
         X_train_subset, X_test_subset, y_train_subset, y_test_subset = train_test_split(
             X_subset, y_subset, test_size=0.2, random_state=42)


         rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
         rf_model.fit(X_train_subset, y_train_subset)
         rf_predictions = rf_model.predict(X_test_subset)


         print("Random Forest:")
         print("Accuracy:", accuracy_score(y_test_subset, rf_predictions))
         print("Classification Report:\n", classification_report(y_test_subset, rf_predictions))
```

```
Random Forest:
Accuracy: 0.9741741741741742
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      1512
           1       0.87      0.84      0.86       153

    accuracy                           0.97      1665
   macro avg       0.93      0.92      0.92      1665
weighted avg       0.97      0.97      0.97      1665
```

Table 2.2 Transactional Random Forest Accuracy

XGBoost:

```
In [19]: import xgboost as xgb

         xgb_model = xgb.XGBClassifier(n_estimators=100, random_state=42)
         xgb_model.fit(X_train_subset, y_train_subset)
         xgb_predictions = xgb_model.predict(X_test_subset)

         print("\nXGBoost:")
         print("Accuracy:", accuracy_score(y_test_subset, xgb_predictions))
         print("Classification Report:\n", classification_report(y_test_subset, xgb_predictions))

         #there are 100 decision trees, based on n_estimators value
         for x in range(10):
           plt.figure(figsize=(20, 10))
           xgb.plot_tree(xgb_model, num_trees=x)
           plt.show()
```

```
XGBoost:
Accuracy: 0.9747747747747748
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      1512
           1       0.88      0.84      0.86       153

    accuracy                           0.97      1665
   macro avg       0.93      0.91      0.92      1665
weighted avg       0.97      0.97      0.97      1665

<Figure size 2000x1000 with 0 Axes>
```

```
<Figure size 2000x1000 with 0 Axes>
```

```
<Figure size 2000x1000 with 0 Axes>
```

Table 2.3.Fig 2.4 Transactional XGBoost Accuracy & Decision trees

Next, we applied traditional machine learning techniques to Clickstream data, using algorithms to analyze user interaction metrics such as clicks, keystrokes, and session duration. This analysis helped us identify behavioral anomalies, like erratic navigation or irregular click intervals, which may indicate fraudulent activity.

```
[ ] df.head()
```

	uid	session_id	timestamp	event_type	screen_x	screen_y	distance	interval	speed	isFraud
0	42596810-60fd-4b69-8fe2-9904bc848595	user_1	1.660000e+12	Mouse Click	864.00000	248.00000	0.000000	0	NaN	0
1	481dcd8f-c948-48f2-9412-2e448b799e89	user_1	1.660000e+12	Mouse Click	869.57605	322.76715	74.974789	2864940	0.000026	0
2	0b166cf8-4c01-4f20-af40-773ad7121a51	user_1	1.660000e+12	Scroll Up	1611.00000	799.00000	881.196460	17148793	0.000051	0
3	3298efab-849a-4c4f-b87a-a3c3fbff0799	user_1	1.660000e+12	Key Press	1748.00000	913.00000	178.227383	79663262	0.000002	0
4	bbf34e30-3478-4a5a-8e0b-0af5fdc590e1	user_1	1.660000e+12	Scroll Up	448.00000	485.00000	1368.643124	75035787	0.000018	0

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
[ ] target = df['isFraud']
    df_features = df.drop(columns=['isFraud'])
    from sklearn.preprocessing import LabelEncoder
    le=LabelEncoder()
    for x in df_features.columns:
      if df_features[x].dtype == 'object':
        df_features[x]=le.fit_transform(df[x])

    #while training the dataset using XGBoost and SVM, object datatype is not accepted
```

```
[ ] df_features
```

	uid	session_id	timestamp	event_type	screen_x	screen_y	distance	interval	speed
0	269	0	1.660000e+12	1	864.00000	248.000000	0.000000	0	NaN
1	292	0	1.660000e+12	1	869.57605	322.767150	74.974789	2864940	0.000026
2	46	0	1.660000e+12	3	1611.00000	799.000000	881.196460	17148793	0.000051
3	215	0	1.660000e+12	0	1748.00000	913.000000	178.227383	79663262	0.000002
4	732	0	1.660000e+12	3	448.00000	485.000000	1368.643124	75035787	0.000018

Table 2.4 Clickstream Dataset

Random Forest:

```
[ ] #Given the features of the dataset, outlier detection is not necessary.
    import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestClassifier
    from xgboost import XGBClassifier
    from sklearn.svm import SVC
    from sklearn.metrics import accuracy_score, classification_report
    X=df_features
    X['speed'].fillna(0)
    y=target
    X_train_subset, X_test_subset, y_train_subset, y_test_subset = train_test_split(
        X,y, test_size=0.2, random_state=42)


    rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_model.fit(X_train_subset, y_train_subset)
    rf_predictions = rf_model.predict(X_test_subset)

    print("Random Forest:")
    print("Accuracy:", accuracy_score(y_test_subset, rf_predictions))
    print("Classification Report:\n", classification_report(y_test_subset, rf_predictions))
```

```
Random Forest:
Accuracy: 0.96
Classification Report:
               precision    recall  f1-score   support

           0       0.94      0.99      0.96       104
           1       0.99      0.93      0.96        96

    accuracy                           0.96       200
   macro avg       0.96      0.96      0.96       200
weighted avg       0.96      0.96      0.96       200
```

Table 2.5 Clickstream Random Forest Accuracy

XGBoost:

```
[ ] import xgboost as xgb

    xgb_model = xgb.XGBClassifier(n_estimators=100, random_state=42)
    xgb_model.fit(X_train_subset, y_train_subset)
    xgb_predictions = xgb_model.predict(X_test_subset)

    print("\nXGBoost:")
    print("Accuracy:", accuracy_score(y_test_subset, xgb_predictions))
    print("Classification Report:\n", classification_report(y_test_subset, xgb_predictions))
```

```
XGBoost:
Accuracy: 0.985
Classification Report:
               precision    recall  f1-score   support

           0       0.97      1.00      0.99       104
           1       1.00      0.97      0.98        96

    accuracy                           0.98       200
   macro avg       0.99      0.98      0.98       200
weighted avg       0.99      0.98      0.98       200
```

Table 2.6 Clickstream XGBoost Accuracy

SVM Model:

```
[ ] svm_model = SVC(random_state=42)
    svm_model.fit(X_train_subset, y_train_subset)
    svm_predictions = svm_model.predict(X_test_subset)

    print("\nSVM:")
    print("Accuracy:", accuracy_score(y_test_subset, svm_predictions))
    print("Classification Report:\n", classification_report(y_test_subset, svm_predictions))
```

```
SVM:
Accuracy: 0.52
Classification Report:
               precision    recall  f1-score   support

           0       0.52      1.00      0.68       104
           1       0.00      0.00      0.00        96

    accuracy                           0.52       200
   macro avg       0.26      0.50      0.34       200
weighted avg       0.27      0.52      0.36       200
```

Table 2.7 Clickstream SVM Accuracy

Finally, we moved on to User Profiling data, using ML algorithms to analyze attributes such as email validation status, breach history, and user demographics. By identifying inconsistencies or suspicious traits in user profiles, we could improve fraud detection accuracy by distinguishing genuine users from potential fraudsters. Each of these efforts laid the groundwork for a more comprehensive, integrated fraud detection model.

```
[ ] df1.head()
```

	user_id	name	age	email_id	email_domain	email_breach_history	email_isValid	email_domainValidation	email_Microsoft	email_LinkedIn	email_Meta	email_DisposableId	email_emailAge	phone_nu
0	user_1	Ava Smith	65	cygsgogo@gmail.com	gmail	4	True	True	True	True	False	False	1	776939
1	user_2	James Rodriguez	63	hiducwxc@aol.com	aol	1	True	True	False	False	True	False	2	476402
2	user_3	Sophia Miller	42	pebhatpz@hotmail.com	hotmail	0	True	True	True	False	True	False	2	841645
3	user_4	Michael Jones	30	igbqyexn@hotmail.com	hotmail	0	True	True	False	False	False	False	5	721096
4	user_5	James Jones	51	rcatufxq@aol.com	aol	4	True	True	True	False	False	False	2	322692

Table 2.8 User Profile Dataset

Random Forest:

```
#Given the features of the dataset, outlier detection is not necessary.
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
X=df_features1
y=target1
X_train_subset, X_test_subset, y_train_subset, y_test_subset = train_test_split(
    X,y, test_size=0.2, random_state=42)


rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_subset, y_train_subset)
rf_predictions = rf_model.predict(X_test_subset)

print("Random Forest:")
print("Accuracy:", accuracy_score(y_test_subset, rf_predictions))
print("Classification Report:\n", classification_report(y_test_subset, rf_predictions))
```

```
Random Forest:
Accuracy: 0.75
Classification Report:
               precision    recall  f1-score   support

           0       0.75      0.67      0.71         9
           1       0.75      0.82      0.78        11

    accuracy                           0.75        20
   macro avg       0.75      0.74      0.74        20
weighted avg       0.75      0.75      0.75        20
```

Table 2.9 User Random Forest Accuracy

XGBoost:

```
import xgboost as xgb

xgb_model = xgb.XGBClassifier(n_estimators=100, random_state=42)
xgb_model.fit(X_train_subset, y_train_subset)
xgb_predictions = xgb_model.predict(X_test_subset)

print("\nXGBoost:")
print("Accuracy:", accuracy_score(y_test_subset, xgb_predictions))
print("Classification Report:\n", classification_report(y_test_subset, xgb_predictions))
```

```
XGBoost:
Accuracy: 0.75
Classification Report:
               precision    recall  f1-score   support

           0       0.70      0.78      0.74         9
           1       0.80      0.73      0.76        11

    accuracy                           0.75        20
   macro avg       0.75      0.75      0.75        20
weighted avg       0.76      0.75      0.75        20
```

Table 2.10 User XGBoost Accuracy

SVM:

```
[ ] svm_model = SVC(random_state=42)
    svm_model.fit(X_train_subset, y_train_subset)
    svm_predictions = svm_model.predict(X_test_subset)

    print("\nSVM:")
    print("Accuracy:", accuracy_score(y_test_subset, svm_predictions))
    print("Classification Report:\n", classification_report(y_test_subset, svm_predictions))
```

```
SVM:
Accuracy: 0.55
Classification Report:
               precision    recall  f1-score   support

           0       0.50      0.33      0.40         9
           1       0.57      0.73      0.64        11

    accuracy                           0.55        20
   macro avg       0.54      0.53      0.52        20
weighted avg       0.54      0.55      0.53        20
```

Table 2.11 User SVM Accuracy

OUTPUT:

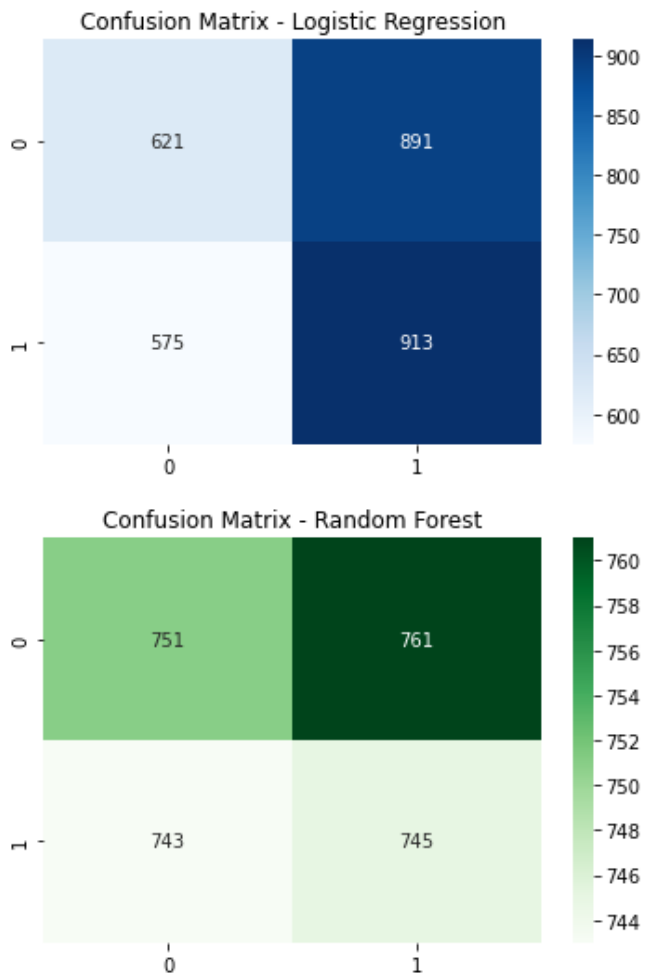

Fig 2.5 Confusion matrix

2.5 FEATURE EXTRACTION

Feature Engineering and Model Preparation

To enhance the effectiveness of our fraud detection system, the following steps are taken to process raw data into meaningful insights and create an optimized model for identifying fraudulent behaviour.

Feature Extraction

The initial step in processing raw data involves feature extraction, where key aspects of user interactions are captured and transformed into measurable attributes. This phase focuses on identifying critical behavioural markers that are indicative of potential fraud. Important features derived from clickstream and telemetry data may include:

- Session Duration: The total time a user spends per session, which may reflect typical versus unusual usage patterns.
- Click Intervals: The time gaps between consecutive clicks, which can highlight erratic or robotic behaviour.
- Device Details: Information on device type, operating system, and browser, which aids in detecting suspicious access from unknown devices.

From the total features extracted (denoted as X), only a subset (Y) of highly relevant features is selected based on their impact on model accuracy. This selection improves model performance by reducing data complexity, making the model more efficient and interpretable.

FINAL DATAFRAMES

```
[ ] df1.head()
```

	age	email_domain	email_breach_history	email_isValid	email_domainValidation	email_Microsoft	email_LinkedIn	email_Meta	email_DisposableId	email_emailAge	phone_number	phone_carrier	phone_disposableSim	phone_phoneAge	cluster_id
0	65	3	4	1	1	1	1	0	0	1	7769391493	1	0	5	0
1	63	0	1	1	1	0	0	1	0	2	4764029916	3	0	1	1
2	42	4	0	1	1	1	0	1	0	2	8416451039	3	0	1	1
3	30	4	0	1	1	0	0	0	0	5	7210961045	5	1	3	0
4	51	0	4	1	1	1	0	0	0	2	3226928096	5	1	5	0

```
[ ] df2.head()
```

	timestamp	event_type	screen_x	screen_y	distance	interval	speed	isFraud
0	0	1	509	167	0	0	834	0
1	0	1	511	248	20	2864940	322	0
2	0	3	771	683	742	17148793	364	0
3	0	0	783	717	95	79663262	96	0
4	0	3	253	435	870	75035787	294	0

```
[ ] df3.head()
```

	type	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	isFraud	isFlaggedFraud	user_id
0	1	65259	0	0	12939	0	0	13
1	0	28326	44939	31614	20340	0	0	20
2	4	75703	35493	0	0	1	0	41
3	0	36255	36727	24202	20884	0	0	33

Table 2.12 Feature Extraction

2.6 DIMENSIONALITY REDUCTION

After feature extraction, dimensionality reduction techniques are applied to further streamline the data and minimize redundancy. Reducing the feature space from Y to Z helps eliminate noise, leading to faster and more accurate model training. Methods like Principal Component Analysis (PCA) are utilized to identify and retain only the most informative features while discarding less significant ones. By focusing on essential features, the model becomes more adept at detecting subtle patterns indicative of fraud, enhancing both processing speed and detection accuracy.

PRINCIPAL COMPONENT ANALYSIS

```
[ ] from sklearn.decomposition import PCA
    from sklearn.preprocessing import StandardScaler
    import pandas as pd

    def apply_pca(df, n_components):
        X = df.values
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)
        pca = PCA(n_components=n_components)
        X_pca = pca.fit_transform(X_scaled)

        column_names = [f'PC{i+1}' for i in range(n_components)]
        df_pca = pd.DataFrame(X_pca, columns=column_names)

        print(df_pca.head())
        print("Explained Variance Ratio:", pca.explained_variance_ratio_)
        return df_pca
```

```
[ ] df1_pca=apply_pca(df1.drop('cluster_id',axis=1),5)
    df2_pca=apply_pca(df2.drop('isFraud',axis=1),3)
    df3_pca=apply_pca(df3.drop('isFraud',axis=1),3)
```

```
        PC1       PC2       PC3       PC4       PC5
0 -0.598484 -0.245525  0.968213 -2.080251  1.019782
1 -1.168744 -0.555884 -0.757544  0.007978 -1.967364
2 -0.887601 -1.378397  0.980346  0.547600 -1.162644
3 -1.130851  0.981846 -0.233950  0.325635 -1.024002
4 -0.947483  0.671790  0.026427 -1.407618 -0.021607
Explained Variance Ratio: [0.19097749 0.1086123  0.09422061 0.08543547 0.08145796]
        PC1       PC2       PC3
0 -1.210899 -1.214766  0.507298
1 -0.047074 -0.410371  0.638497
2 -1.016139  2.347741  1.576636
3  0.088981  2.293428 -0.194184
4  0.572387  0.484106  1.217513
Explained Variance Ratio: [0.29131278 0.26659979 0.16898525]
        PC1       PC2       PC3
0 -0.175350  1.355956  0.581325
1  2.969920 -1.823908  0.187280
2 -0.283734 -0.212209  0.440842
3  2.435922 -1.055331  0.058860
4 -0.547177 -0.293131  0.338037
Explained Variance Ratio: [0.31497777 0.19332386 0.14361563]
```

Table 2.13 Dimensionality Reduction

2.7 CUSTOM CLUSTERING MODEL

A detailed overview of the model is extremely essential for the explainability part of it.

The approach used in most fraud detection models was the lack of data and approach hence in the case of our dataset we apply “Hybrid Clustering and Neural Network Model for Fraud Detection”. This involves using a Custom Clustering Model that is created using a neural network and its hidden layers

Libraries and Models Used :

Implements this hybrid fraud detection model using many Python libraries that allow data manipulation, visualization, model building, and performance evaluation in this system. All these libraries performed crucially steps in pre-processing the dataset, model design, and interpretation of results.

Visualization :

Exploratory Data Analysis: This platform allowed for the comprehensive EDA to be performed using a powerful combination of libraries available, namely Matplotlib and Seaborn. It enabled some deep visual insights into the intricacies of complex patterns and the relationships inside the transaction dataset. The systematic step was there, but from there, the fine anomalies and distinct patterns within the dataset that otherwise might not have been in view with traditional statistical analysis could be duly identified.

Matplotlib: Some complex visualizations of graphics were laid upon a foundational block of this library-from detailed distribution plots up to nuanced fraud score visualizations and advanced model performance metrics. Important indicators for what the given data looked like and how good the model was at distinguishing fraudulent transactions from legitimate ones included ROC curves and confusion matrices. These visualizations were therefore key to explaining just how effective this model was and in which places performance could be optimized.

Seaborn: This advanced visualization library took the analytical process from Matplotlib to another level altogether by creating even more refined and informative visualizations. Complex interdependencies between variables were represented through matrices of correlations, while superb histogram plots shed light on subtle distributional patterns. These sophisticated visualization techniques greatly enhanced feature analysis and further solidified the overall exploratory data analysis as it provided deeper insights into some patterns and relationships that lie under the dataset. Seaborn visualizations had an aesthetic appeal and statistical sophistication that brought the complex patterns into a bit more understandable and more interpretable plane, thus making the decisions for the model development process even more informed.

Graphs were created in Python using Matplotlib and seaborn, with standard figure sizes of 15 x 10 inches to maximize clear and readable output. Color palettes ("Set2" and "Set3") were intentionally selected to maximize categorical discrimination without sacrificing visual accessibility.

Data Analyzing Techniques to Provide Graphical Representation:

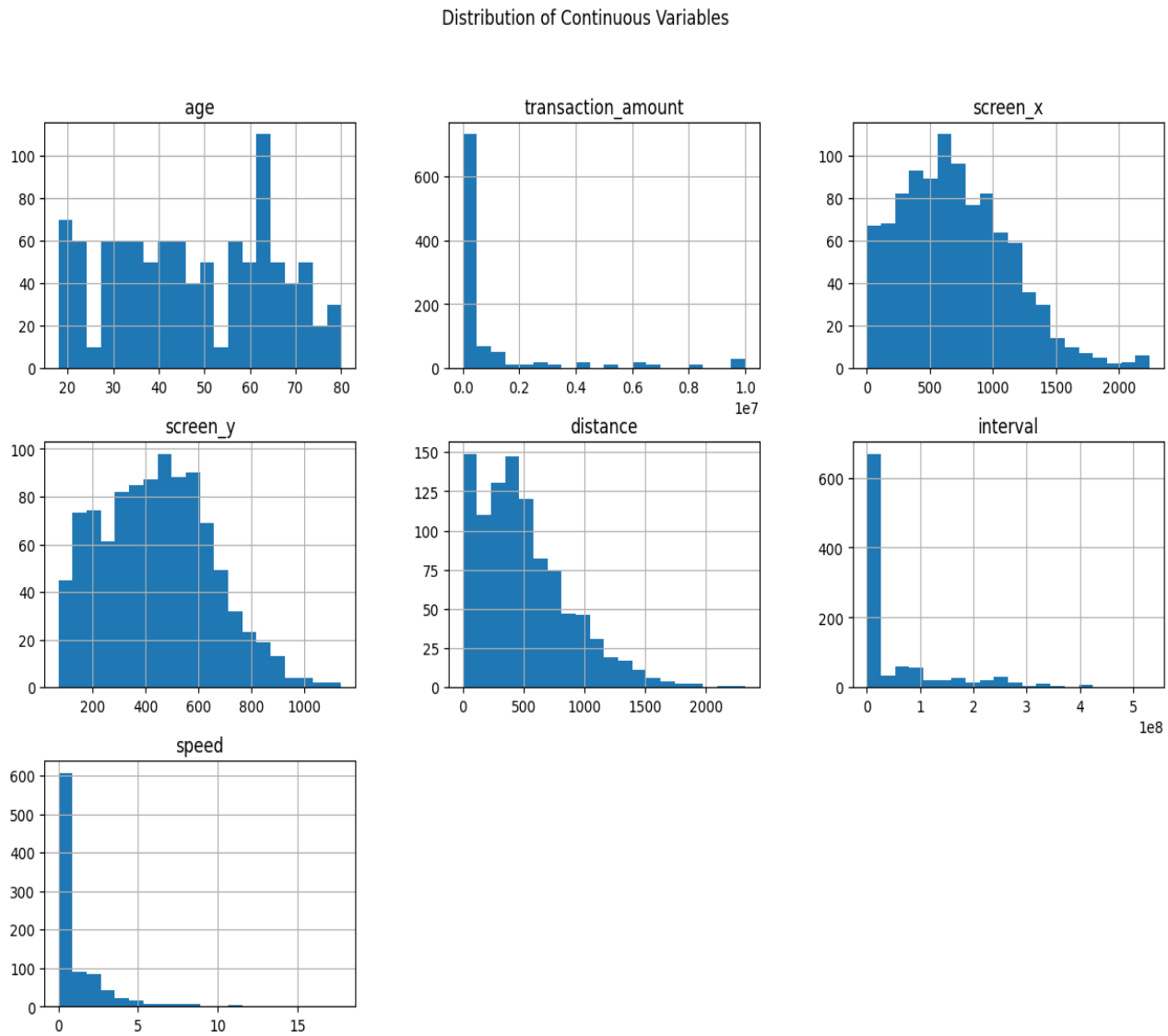

Fig 2.6 Distribution of continuous variables

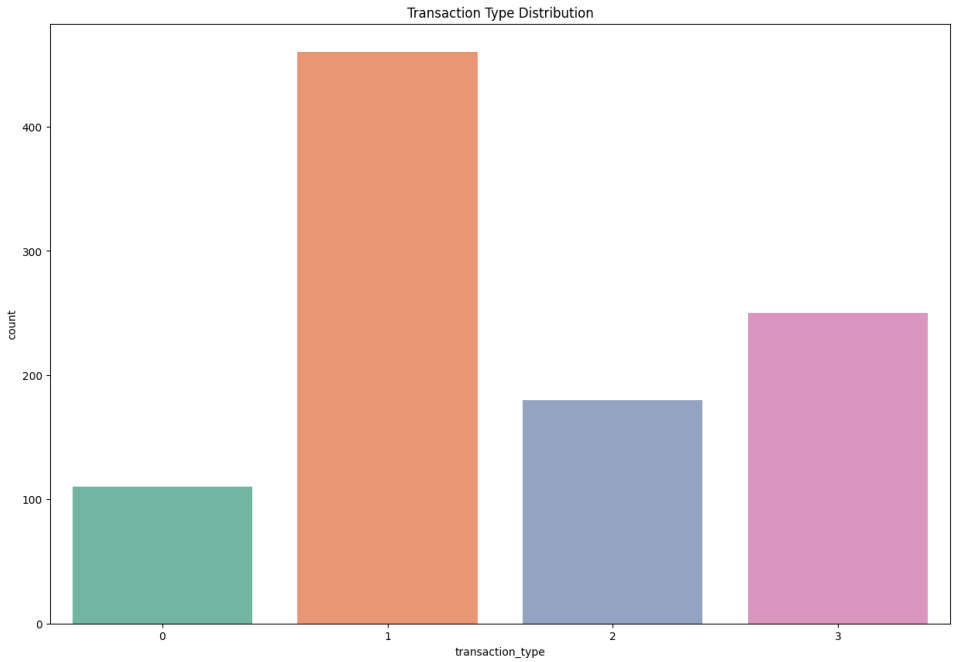

Fig. 2.7 Transaction distribution type

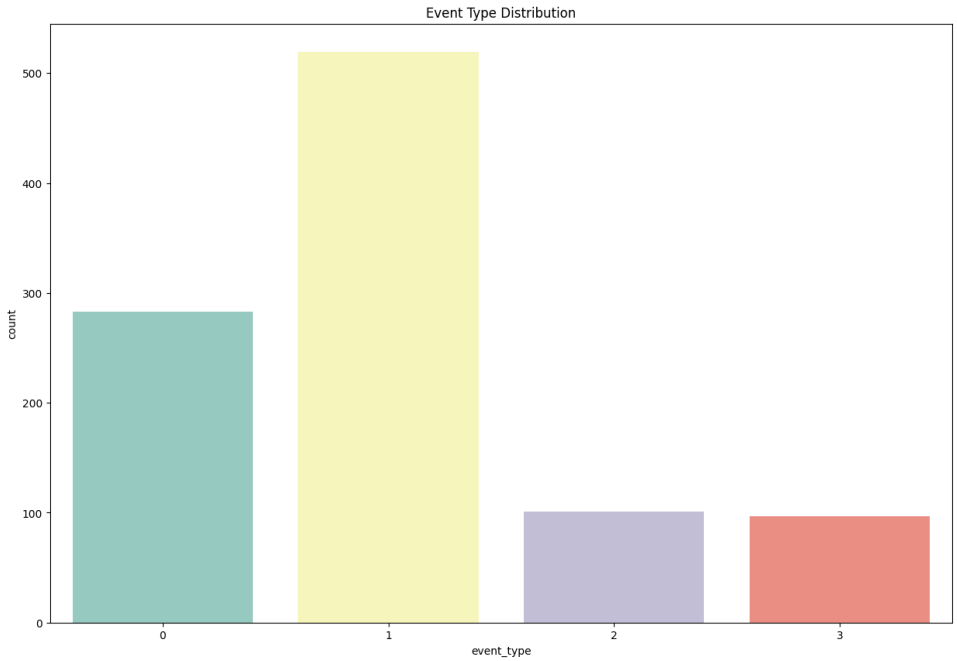

Fig. 2.8 Event distribution type

These distributions provide insight into the following critical issues:

Transactional Patterns: Describe what the transaction distribution indicates about the usage of the system or business patterns:

Event Frequency: What does the event distribution tell me about the behaviour of the

system or user interactions?

Data balance: Discuss implications for subsequent analysis or modelling decisions

Transaction Amount by Fraud Status :

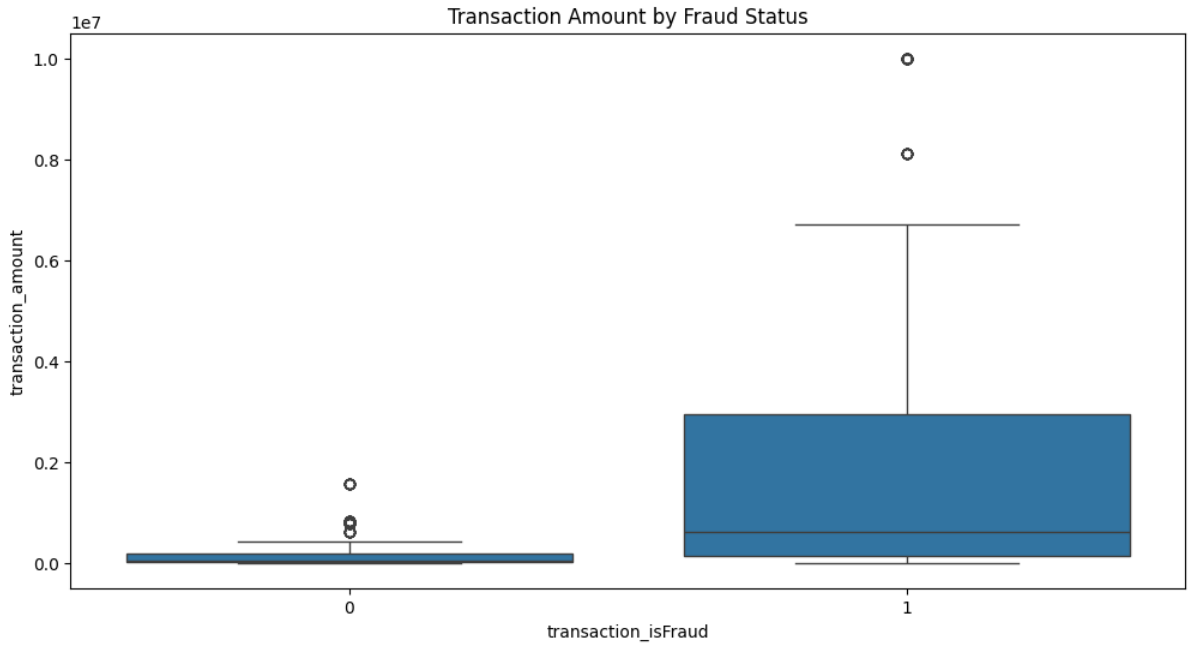

Fig 2.9 Transaction Amount by Fraud Status

Event type by Fraud Status :

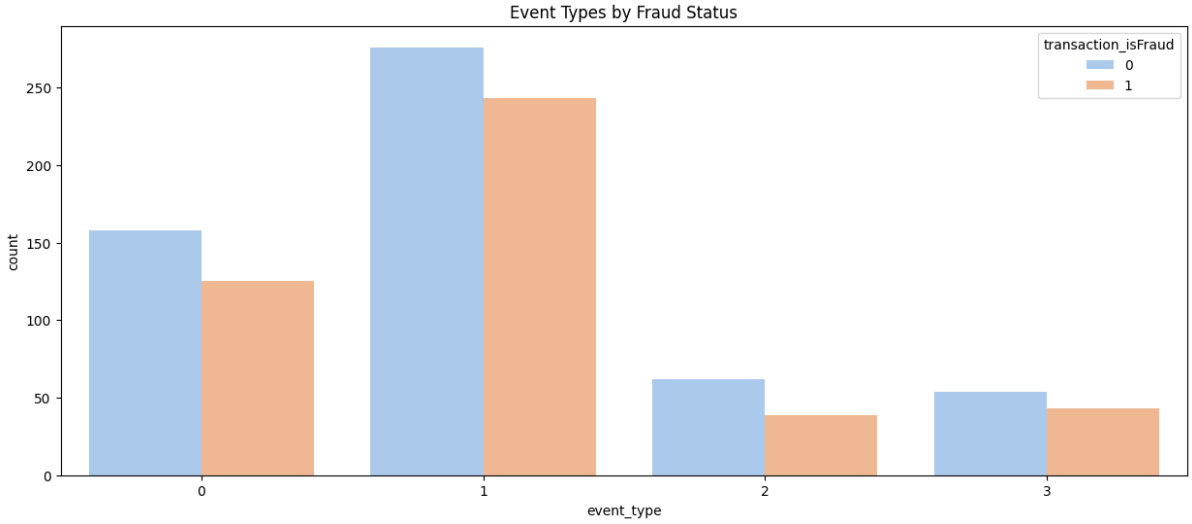

Fig 2.10 Event type by Fraud Status

Both graphs in detail:
Types of Events by Fraud Status (Image 1):
The bar graph displays the distribution of various types of events, categorized according to fraud status, where,
Blue bars indicate non-fraudulent transactions, i.e., 0
Salmon/pink bars indicate fraudulent transactions, i.e., 1
Important observations:
Event type 1 happens the most for both fraudulent and non-fraudulent transactions
Event type 0 has the next frequency distribution
Event types 2 and 3 have relatively lower frequencies
Non-fraudulent transactions are by a very narrow margin larger than fraudulent ones
Transaction Amount by Fraud Status (Figure 2)
This box plot shows the comparison between fraudulent and non-fraudulent transaction amounts:
0 on the x-axis is non-fraudulent transactions, 1 is fraudulent transactions

Visualize the explained variance ratio of the PCA components:

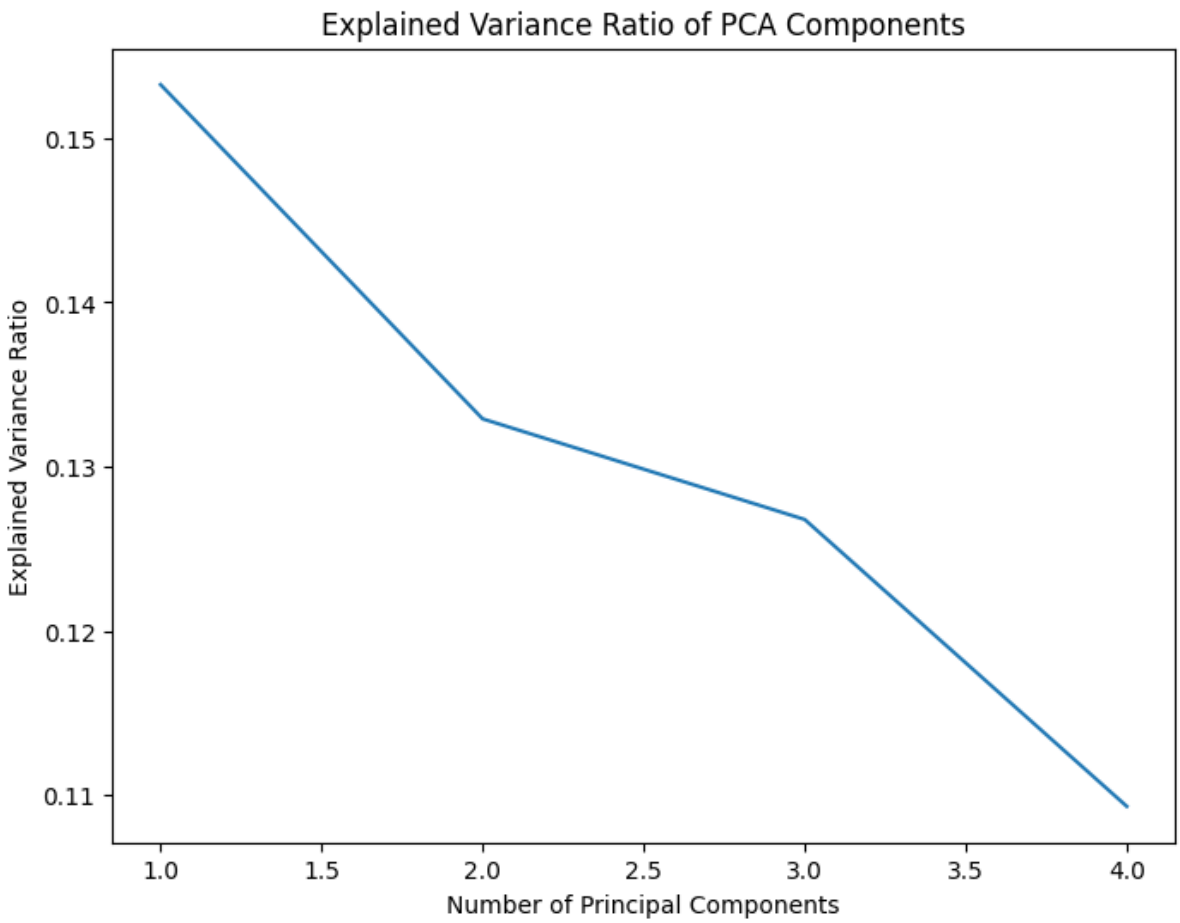

Fig 2.11 Explained Variance Ratio of PCA Components

These visualizations collectively indicate that fraud transactions seem to be larger in amounts and have more variability in transaction size, but at the same time, they provide patterns related to events associated with fraud, useful for the fraud detection system and risk assessment.

Machine Learning and Model Building

KMeans: The KMeans clustering algorithm from sklearn.cluster was used to identify anomalies by grouping transactions into clusters. KMeans was chosen for its simplicity and efficiency in handling high-dimensional data, as well as for being a commonly used technique in fraud detection for initial anomaly screening

Agglomerative Clustering: Agglomerative Clustering is used to understand transactional hierarchical relationships. It does not require the antecedent definition of the count of clusters and is valuable for identifying faint, subtle patterns that potentially indicate fraud. Agglomerative Clustering differs from K-Means, as K-Means partitions data only in spherical shapes, whereas Agglomerative Clustering generates a cluster hierarchy, which allows for further nested patterns to assist in distinguishing between legitimate and anomalous transactions.

GridSearchCV: Used for hyperparameter tuning to optimize the neural network model's performance. GridSearchCV enabled automated searching over a predefined set of hyperparameters, ensuring that the best configuration was selected based on cross-validation performance.

MLPClassifier: The MLP classifier from sklearn.neural_network was taken as the core neural network for the classification task. MLPClassifier is a feed-forward neural network that can run with several different hidden layers and activation functions making it quite well-suited to perform non-linear modeling in data. For this model, ReLU activation in the hidden layers has been used to increase non-linearity and Softmax in the output layer accounting for multi-class probabilistic classification.

Neural Network Model

The architecture was set up in a way that built upon the insights offered by the clusters. It included a feed-forward architecture with two hidden layers aimed at capturing all the non-linear interactions in the dataset. The ReLU activation function was appropriate for the hidden layers since it did not produce vanishing gradients and is computationally very efficient. Softmax outputs were necessary for the final layer in the network for probabilistic results. Regularization through an L2 penalty with an alpha of 0.1 was added to prevent overfitting; key hyperparameters, such as the hidden

layer sizes, learning rate, and maximum number of iterations, were optimized using grid search. This neural network design enabled the model to observe complex patterns in the data and sharpen the effect of anomaly detection being performed through clustering.

The architecture combines the features reduced by PCA along with the clustering information within the framework of an MLP. The PCA components are concatenated with the binary cluster assignments obtained from both the k-means and agglomerative clustering algorithms in the input layer for the neural network, thus enriching the representation of features to capture not only linear patterns but also non-linear patterns in data.

To determine the optimal architecture of the network for the task, exhaustive grid search cross-validation was carried out to determine the best possible configurations for the set of hyperparameters.

Hidden layer architectures: [(10, 5), (50, 25), (50, 25, 10)]

Activation functions: ReLU and hyperbolic tangent

Learning rate schedules: constant and adaptive

Regularization strength (α): [0.001, 0.01, 0.1]

The network makes use of L2 regularization for which the regularization strength has been optimized during the grid search process. The final layer is fed with a softmax activation function to output class probabilities of fraud. Ensemble the predictions from this model with that of clustering labels using weighted ensemble .

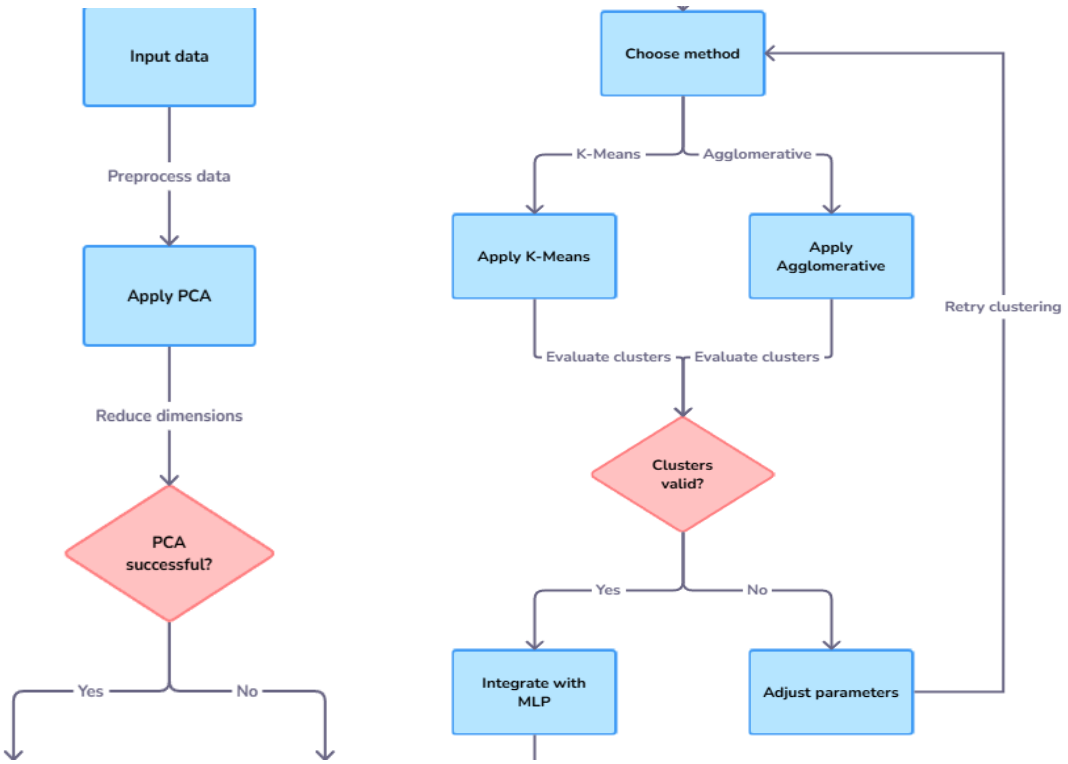

Fig 2.12 Clustering Flowchart

The validation of the clustering model output is key to finding and tuning the parameters .By using K-Means and Agglomerative Clustering, the model will cross-validate and cross-check clusters, thus ensuring that there is strong outlier and anomaly detection. These two kinds of clustering models help identify fraud more completely: using general patterns with K-Means, and nested, complex patterns with Agglomerative Clustering.

Agglomerative clustering is based on distance calculations. For example, Euclidean or Manhattan distances between data points are used to construct clusters. NaN values interfere with such calculations as there will be undefined values in the data, which can make it impossible to calculate a distance.

It would be surprising just to apply this algorithm in the raw state without preprocessing data with missing values imputed, removed, or mean-substituted before running the algorithm, because accurate similarity assessments can only be done after proper preprocessing of the data and meaningful hierarchies can only be obtained from the results of the clustering.

Plotted Graphs:

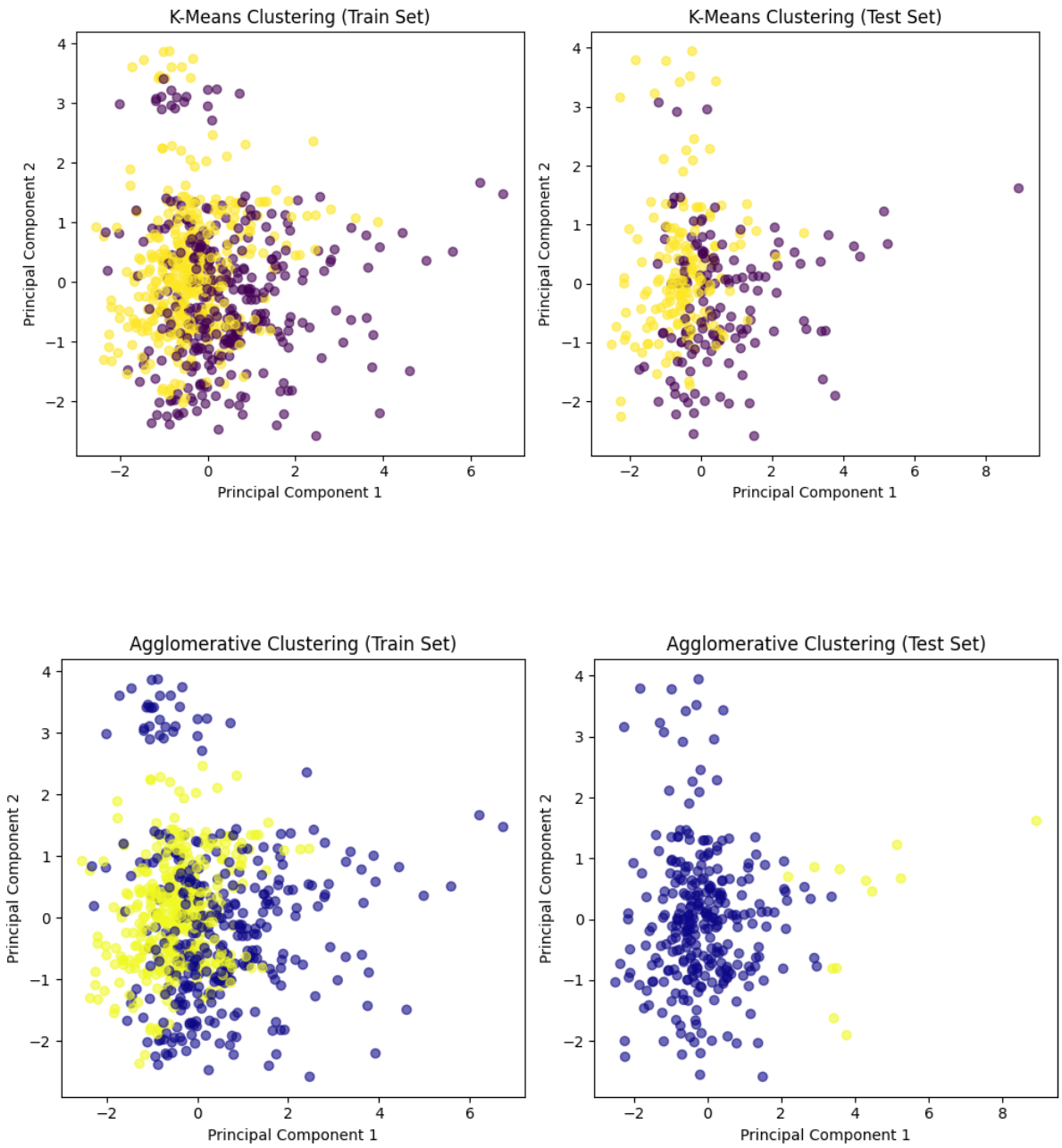

Fig. 2.13 Plotted Graphs

The complexity of the data set only requires higher analyses to bring meaningful information and trends from the data. Data is multifaceted and requires in-depth analysis in search of underground relationships and trends that may be hidden between various data points.

Hyperparameter Tuning:

Hyperparameter tuning for MLP has paramount importance in fraud detection to optimise its performance:

hidden_layer_sizes: It determines the number and size of hidden layers. The choice of such parameters determines whether your model is capable enough to sense complex patterns. In most cases, you'd want a balance of capacity with the risk of overfitting to noise.

Activation: It defines how neurons transform input data. ReLU is common for faster convergence and can help pick up non-linear relationships in fraud patterns.

learning_rate: The weights are learned, that are corrected for their performance, and this prevents unstable training and better detection of rare fraud events.

Alpha: This is a regularization parameter that prevents overfitting by penalizing large weights. And it becomes very crucial while handling imbalanced data in fraud detection.

max_iter: That will give the number of maximum iterations to be required for the training of the model. Taking proper iterations, the model can learn well from the data.

Tol: Tolerance for stopping criterion. A low tolerance prevents getting stuck too early, and as a consequence, proper model training toward detection of fraud.

These parameters balance model complexity, prevent overfitting, and also correctness in fraud Score.

```
GridSearchCV
best_estimator_: MLPClassifier
MLPClassifier
MLPClassifier(alpha=0.1, hidden_layer_sizes=(50, 25), max_iter=2000,
              random_state=42)
```

```
# Get the best model from GridSearchCV
best_nn_classifier = grid_search.best_estimator_

# Fit the best model on the training data
best_nn_classifier.fit(X_train_combined, y_train)
```

```
MLPClassifier
MLPClassifier(alpha=0.1, hidden_layer_sizes=(50, 25), max_iter=2000,
              random_state=42)
```

.

Neural Network Probabilities:

y_prob_train and y_prob_test store the probability of fraud (class 1) predicted by the neural network. These probabilities are weighted at 70% of the final fraud score.

Clustering Labels: K-means and agglomerative clustering labels (kmeans_labels_train, agg_cluster_labels_train) represent fraud likelihood based on cluster membership. These

are weighted at 15% each.

Fraud Score Calculation: The fraud score is a weighted sum of these three components: 70% from the neural network and 15% each from clustering labels.

Binary Classification: The fraud score is thresholded at 0.5 to classify transactions as fraud (1) or not fraud (0), producing y_pred_train and y_pred_test for training and testing datasets.

This method leverages both supervised (neural network) and unsupervised (clustering) techniques to improve fraud detection accuracy.

train_fraud_score = (0.7*y_prob_train) + (0.15*kmeans_labels_train) + (0.15*agg_cluster_labels_train)

test_fraud_score = (0.7 * y_prob_test) + (0.15 * kmeans_labels_test) + (0.15 * agg_cluster_labels_test)

2.8 RESULTS & OBSERVATIONS

The performance of the model is evaluated in two main metrics: the classification report and the confusion matrix.

Custom Classifier Outputs:

Classification Report: It provides precision, recall, F1 score, and accuracy both for training and testing datasets. These metrics will assess how fraud is detected by the model by balancing false positives and false negatives.

Confusion Matrix: the counts of true positives, true negatives, false positives and false negatives to get a better understanding of how much the model is erring while it predicts fraud.

The print statements are also outputting these results for both datasets, which helps in testing how well the model generalizes and does the job on unseen data, but, is very important for fraud detection in real applications.

Classification Report Output:

```
Training Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.91      0.92       386
           1       0.90      0.91      0.90       314

    accuracy                           0.91       700
   macro avg       0.91      0.91      0.91       700
weighted avg       0.91      0.91      0.91       700

Training Confusion Matrix:
 [[353  33]
 [ 28 286]]

Testing Classification Report:
              precision    recall  f1-score   support

           0       0.73      0.91      0.81       164
           1       0.85      0.60      0.70       136

    accuracy                           0.77       300
   macro avg       0.79      0.76      0.76       300
weighted avg       0.79      0.77      0.76       300

Testing Confusion Matrix:
 [[150  14]
 [ 55  81]]
```

Table 2.14 Custom Model Accuracy

Sample Fraud Scores for Testing Set:

[1.54130807e-01 6.99999998e-01 1.95814556e-01 4.82231657e-07
5.72862320e-01 5.34046925e-01 2.03491743e-01 4.82301922e-01
4.29325449e-02 1.50661002e-01]

Testing Confusion Matrix:

[[150 14]
[55 81]]

Print accuracy, F1-score, precision, and recall for both datasets so that you easily compare your training and testing results. That would decide whether the model is generalizing or over-fitting

Best Hyperparameters:

The best hyperparameters the GridSearchCV has uncovered are printed. It has the information about what the configuration is that maximizes model performance. These metrics would render a complete view of the precision and consistency of the model in terms of fraud detection. For balancing misclassifications to hit the optimal mark in fraud

detection, precision and recall would prove quite essential, while accuracy would amount to an overall success rate.

Output:
Training Accuracy: 91.29%
Testing Accuracy: 77.00%
Training F1-Score: 0.90
Testing F1-Score: 0.70
Training Precision: 0.90
Testing Precision: 0.85
Training Recall: 0.91
Testing Recall: 0.60

Best Hyperparameters for Neural Network: {'activation': 'rel', 'alpha': 0.1, 'hidden_layer_sizes': (50, 25), 'learning_rate': 'constant', 'max_iter': 2000, 'tol': 0.0001}

```
Training Accuracy: 91.29%
Testing Accuracy: 77.00%
Training F1-Score: 0.90
Testing F1-Score: 0.70
Training Precision: 0.90
Testing Precision: 0.85
Training Recall: 0.91
Testing Recall: 0.60

Best Hyperparameters for Neural Network: {'activation': 'relu', 'alpha': 0.1, 'hidden_layer_sizes': (50, 25), 'learning_rate': 'constant', 'max_iter': 2000, 'tol': 0.0001}
```

Chapter 3
Future Scope

3.1 PROTOTYPE CONSTRAINTS

The research faces specific constraints related to data accessibility and privacy:

- **Data Imbalance and Privacy Limitations**: Fraudulent transactions are naturally rare, resulting in an imbalanced dataset. Additionally, the availability of comprehensive clickstream and user persona data is limited by privacy regulations, which restricts access to certain types of behavioural information. This constraint necessitates careful handling of data and the implementation of privacy-preserving techniques, ensuring compliance with data protection standards while maintaining model accuracy.

3.2 PAYFLOW INTERFACE

Currently, the fraud detection system uses a temporary interface built with Gradio to simulate transaction analysis. This interface gathers user inputs like email, phone number, transaction type, amount, and differences in destination and original data, and processes them using a pre-trained XGBoost model. The prediction results are provided in real time in a simple and interactive format.

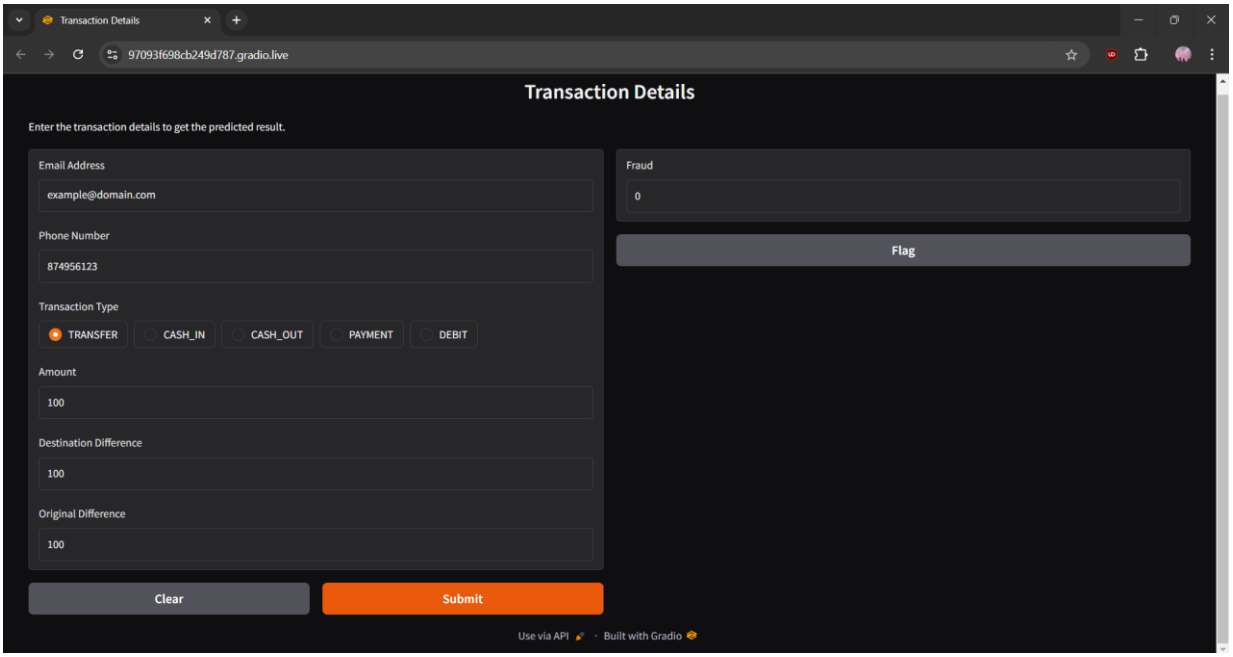

Table 2.14 Demo Payflow Interface

In the future, this system will evolve into a more dynamic, data-rich environment capable of harvesting real-time data from transactional records, clickstream data, and user profiling. By integrating advanced data sources, the system will be able to monitor and analyze transaction patterns on the fly, offering deeper insights into user behaviour and enhancing fraud detection accuracy. This transition will involve incorporating streaming data processing technologies and user-specific profiling to provide tailored fraud prevention measures that adapt to evolving transaction patterns and trends.

3.3 FUTURE SCOPE

The future scope of this fraud detection system holds significant potential for expansion and impact, encompassing both technical development and commercialization opportunities. One of the primary future advancements involves creating a dedicated Payflow frontend page, which would serve as a real-time interface to capture and process transactional, clickstream, and user data. This page would enable comprehensive data harvesting as users interact with the system, providing a rich, live feed of behavioural patterns and transactional details directly into the fraud detection engine. By embedding real-time monitoring capabilities, this Payflow interface would not only streamline data collection but also ensure immediate analysis, allowing for rapid detection of suspicious activities as they occur.

Beyond developing the frontend interface, another critical focus is on training and optimizing the model using proprietary clickstream data from financial institutions and companies. Company-specific clickstream data is often far more detailed and complex than standard datasets, containing a wealth of unique insights into user behaviour, session patterns, and interaction anomalies. By training the model on such high-resolution data, the fraud detection system can gain a much deeper understanding of legitimate versus fraudulent activity, tailored to the particular context of each organization. This customized approach enhances accuracy, allowing the system to identify subtle indicators of fraud that may go unnoticed by traditional models.

Commercialization is a major component of the system’s future scope. As the solution matures, it can be marketed to fintech companies, banks, and financial institutions looking to bolster their fraud prevention strategies. These organizations are increasingly at risk due to the rise in digital transactions, and they need advanced, adaptable tools to safeguard their operations and customer accounts. By integrating this fraud detection system, financial institutions could significantly reduce the risk of fraud, fostering greater trust among their user base and protecting their financial assets. The model's adaptability to organization-specific data and fraud trends makes it a highly valuable asset in the current digital finance landscape.

Furthermore, as the model continues to evolve, it can be enhanced with continuous learning capabilities, allowing it to adapt to emerging fraud tactics and new types of behavioural anomalies over time. This adaptability would ensure that the system remains effective as fraud patterns evolve, offering long-term resilience and value to clients. Future

versions of this system could also incorporate additional data sources, such as social media activity or phone-based user verification metrics, to further strengthen the model's predictive accuracy and contextual understanding of user behaviour.

Ultimately, this system's future scope envisions a powerful, adaptable, and real-time fraud detection tool that is not only technically sophisticated but also commercially viable. It has the potential to become an essential part of the digital finance ecosystem, providing financial institutions with a proactive defence against fraud and enabling them to offer a safer and more reliable experience to their customers. By continually refining and expanding its capabilities, this fraud detection system could become a leading solution in the fight against financial cybercrime.

3.4 TECHNICAL ADVANCEMENTS

Future Work and Advancements

- **Real-Time Payflow Page Integration:** Develop a dedicated frontend page for real-time monitoring and data harvesting of transactional, clickstream, and user profile data. This will enable immediate analysis and detection of fraudulent patterns during live transactions.

- **Advanced Clickstream Data Processing:** Train the model on proprietary clickstream data from financial institutions. Given its complexity and detail, company-specific clickstream data can provide unique insights, allowing the model to identify subtle and company-specific fraud indicators more effectively.

- **Enhanced Model Training with Multimodal Data:** Integrate and process various data types—including transactional, telemetry, and behavioural data—to create a comprehensive fraud detection model. Future versions can further refine this by incorporating additional data sources such as social media interactions and phone-based verification.

- **Machine Learning Model Adaptability:** Build on traditional ML algorithms by exploring custom clustering and advanced anomaly detection techniques, ensuring the model adapts to the specific fraud patterns and data characteristics of each institution.

- **Continuous Learning for Evolving Threats:** Implement a continuous learning mechanism to keep the model up-to-date with emerging fraud tactics and new types of behavioural anomalies. This adaptability will help maintain the model's effectiveness as fraud patterns evolve.

- **User-Specific Fraud Scoring:** Develop a more nuanced fraud scoring system that evaluates the likelihood of fraudulent behaviour for individual users based on a holistic analysis of their behavioural and transactional patterns.

- **Commercialization for Financial Sector:** Prepare the system for commercialization to be offered as a fraud prevention solution to banks, fintech companies, and financial institutions. This will help these organizations safeguard their transactions and enhance customer trust.

- **Enhanced Privacy Measures:** Implement stricter privacy protocols to securely handle sensitive user data (e.g., email, phone interactions) while ensuring compliance with privacy regulations and minimizing data imbalance challenges.

- **Prototyping and Real-World Testing:** Run pilot programs with partner financial institutions to test the system's efficacy in real-world scenarios. These trials can provide valuable feedback to fine-tune the model for maximum accuracy and adaptability in different organizational contexts.

These advancements aim to create a robust, adaptable fraud detection system capable of real-time detection, efficient data handling, and scalable applications across the financial sector making this a huge commercial project.

Chapter 4
Conclusion

4.1 PERSONAL LEARNINGS & GROWTH

Throughout this project, my learning journey has been immense and transformative, spanning multiple areas of technical, analytical, and strategic growth. Working on a complex fraud detection system deepened my understanding of machine learning algorithms and data engineering, particularly in the context of processing multimodal datasets, such as transactional, behavioural, and user profile data. I gained hands-on experience with feature extraction and dimensionality reduction techniques, learning how to streamline large datasets for more effective model training, as well as how to balance data quality with system performance. Diving into clickstream data and telemetry data opened my eyes to the complexities of behavioural analysis and the subtle cues that can differentiate legitimate users from potential fraudsters. Experimenting with various traditional and custom machine learning models, I learned the nuances of model tuning and the importance of selecting the right algorithms for different types of data, as well as the iterative process of optimizing these models for real-world applications.

Beyond technical skills, I developed a strategic mindset on how to structure a project of this scale. Balancing the integration of multiple data sources while maintaining system efficiency required me to think holistically about data flow and to consider the end-to-end lifecycle of fraud detection systems, from data collection to final scoring. Working on this project highlighted the critical role of continuous learning and adaptability, especially as I encountered challenges in handling large, imbalanced datasets and ensuring high accuracy without compromising on speed. It pushed me to stay up-to-date with the latest advancements in fraud detection and machine learning, reinforcing the importance of an agile, growth-oriented approach to technology. Additionally, this project broadened my understanding of industry-specific challenges, including the privacy considerations and regulatory compliance needed in handling sensitive financial data. This experience has not only strengthened my technical skillset but also prepared me for real-world applications where precision, security, and efficiency are paramount. Overall, the project has been a journey of learning, problem-solving, and skill-building that has prepared me to take on even more complex, impactful challenges in the future.

4.2 CONCLUSION

This project represents a significant advancement in the field of online fraud detection, presenting a holistic, data-driven approach that moves beyond the limitations of traditional systems. By integrating multiple data sources—including transactional, clickstream, telemetry, and user persona data—our system aims to capture a comprehensive picture of user behaviour, enabling more precise and accurate detection of fraudulent activities. The step-by-step development process, from individual dataset analysis to custom model implementation, has demonstrated the capability to detect fraud with greater depth and adaptability, significantly reducing false positives and enhancing transaction security.

One of the project's revolutionary aspects lies in its ability to analyze data across multiple domains, providing an integrated view of user interactions that single-dimensional models cannot achieve. This multi-layered approach to fraud detection represents a breakthrough, as it addresses the shortcomings of existing systems that often fail to detect sophisticated fraud patterns. The system's use of advanced feature extraction, dimensionality reduction, and custom clustering techniques allows it to process large datasets efficiently, making it not only accurate but also scalable to real-world applications. As a result, this solution has substantial potential for deployment in high-stakes financial environments where transaction security is paramount.

With further refinement, this project could become a cornerstone technology for fintech companies, banks, and other financial institutions. Its capability to detect fraudulent behaviour based on real-time, high-resolution data makes it a valuable tool for preventing financial losses and enhancing customer trust. By continuously evolving to adapt to emerging fraud tactics, this solution holds the promise of transforming online security standards, contributing to a safer and more reliable digital ecosystem for all stakeholders involved.

In summary, this project has set a new benchmark for fraud detection systems, offering a robust, revolutionary solution that meets the demands of the modern digital landscape. By leveraging holistic data analysis and cutting-edge machine learning techniques, it represents a major leap forward in the ongoing battle against online financial fraud, ensuring that organizations can proactively safeguard their transactions in an increasingly complex digital world.

Appendices

The code for this project, for transaction analysis and the associated fraud detection model, is available in the project's GitHub repository. You can access the full repository to view, clone, or contribute to the codebase.

GITHUB REPOSITORY:
https://github.com/Utkarsh-Dhekale/FraudDetection.git

REFERENCES

1. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9085905
C. Wang and H. Zhu, "Representing Fine-Grained Co-Occurrences for Behavior-Based Fraud Detection in Online Payment Services," in IEEE
Transactions on Dependable and Secure Computing, vol. 19, no. 1, pp. 301-315, 1 Jan.-Feb. 2022, doi 10.1109/TDSC.2020.2991872.

2. https://aclanthology.org/2020.lrec-1.167.pdf
Wang, M.-F., Tsai, M.-F., Jheng, S.-L., and Tang, C.-H. (2012). Social feature-based enterprise email classification without examining email contents.
Journal of Network and Computer Applications, 35(2):770–777.

3. https://ieeexplore.ieee.org/abstract/document/7277068
X. Ruan, Z. Wu, H. Wang and S. Jajodia, "Profiling Online Social Behaviors for Compromised Account Detection," in IEEE Transactions on Information
Forensics and Security, vol. 11, no. 1, pp. 176-187, Jan. 2016, doi: 10.1109/TIFS.2015.2482465.

4. https://doi.org/10.1186/s40854-023-00470-w
Vanini, P., Rossi, S., Zvizdic, E. et al. Online payment fraud: from anomaly detection to risk management. Financ Innov 9, 66 (2023).

5. https://arxiv.org/abs/1808.05329
Zhang, R., Zheng, F., & Min, W. (2018). Sequential Behavioral Data Processing Using Deep Learning and the Markov Transition Field in Online Fraud Detection.
ArXiv. /abs/1808.05329

6. https://dl.acm.org/doi/fullHtml/10.1145/3184558.3186569
Zhang, R., Zheng, F., & Min, W. (2018). Sequential Behavioral Data Processing Using Deep Learning and the Markov Transition Field in Online Fraud Detection.
ArXiv. /abs/1808.05329

7. https://www.sciencedirect.com/science/article/pii/S2772662223000036?via%3Dihub
Jonathan Kwaku Afriyie, Kassim Tawiah, Wilhemina Adoma Pels, Sandra Addai-Henne, Harriet Achiaa Dwamena, Emmanuel Odame Owiredu, Samuel
Amening Ayeh, John Eshun

8.https://www.researchgate.net/publication/228722329_Analysing_clickstream_data_From_anomaly_detection_to_visitor_profiling
Hofgesang, Peter & Kowalczyk, Wojtek. (2005). Analysing clickstream data: From anomaly detection to visitor profiling.

9. https://arxiv.org/pdf/2101.04285
Min, W., Liang, W., Yin, H., Wang, Z., Li, M., & Lal, A. (2021). Explainable Deep Behavioral Sequence Clustering for Transaction Fraud Detection. ArXiv. /abs/2101.04285

10. https://www.sciencedirect.com/science/article/pii/S187705092030065X
Vaishnavi Nath Dornadula, S Geetha, Credit Card Fraud Detection using Machine Learning Algorithms, Procedia Computer Science, Volume
165,2019, Pages 631-641, ISSN 1877-0509

11.https://www.researchgate.net/profile/Winner-Olabiyi/publication/383982767_Real-Time_Transaction_Analysis_The_Importance_of_Analyzing_Transactions_to_Detect_Fraud_Quickly/links/66e347cbfa5e11512cb6e70d/Real-Time-Transaction-Analysis-The-Importance-of-Analyzing-Transactions-to-Detect-Fraud-Quickly.pdf
Winner Olabiyi, Edwin Frank, and Oluwaseyi Joseph. "Real-Time Transaction Analysis: The Importance of Analyzing Transactions to Detect Fraud
Quickly." (2024).

12. https://irep.ntu.ac.uk/id/eprint/47097/
Kian, Ramez, and Hadeel S. Obaid. "Detection of fraud in banking transactions using big data clustering technique customer behaviour indicators."
Journal of applied research on industrial engineering 9.3 (2022): 264-273.

13. E. A. Lopez-Rojas , A. Elmir, and S. Axelsson. "PaySim: A financial mobile money simulator for fraud detection"In The 28th European Modeling and Simulation Symposium-EMSS, Larnaca, Cyprus. 2016 *Dataset*. Kaggle. https://www.kaggle.com/datasets/ealaxi/paysim1

14. Wang, G., Zhang, X., Tang, S., Zheng, H., & Zhao, B. Y. (n.d.). *Unsupervised clickstream clustering for user behaviour analysis*. Department of Computer Science, University of California, Santa Barbara. Retrieved from https://www.cs.ucsb.edu/

15. Hu, Tianrui. (2020). *Detecting bots using stream-based system with data synthesis* (Master's thesis, Virginia Polytechnic Institute and State University). Retrieved from https://scholar.lib.vt.edu/